Particle Flow Filter and Differentiable Particle Filter

Haowu Duan

February 21, 2026

Contents

1 Literature Review

1.1 State-Space Modeling in Financial Markets: The big Picture

Financial systems are driven by latent factors that shape observable market data. Problems such as stochastic volatility, limit order book dynamics, or macroeconomic regime shifts are frequently cast as Hidden Markov Models (HMMs) or continuous state-space models (SSMs). In quantitative finance, Bayesian filtering and smoothing are key tools for recovering these hidden drivers. A central example is *stochastic volatility* modeling, where asset volatility is a drifting latent state. Recent semi-parametric methods employ particle filters to estimate it without rigid parametric forms, yielding more accurate option pricing and risk management [1]. Discrete state-space models, particularly HMMs, excel at *regime detection*. They treat market regimes ("bull" vs. "bear", high vs. low momentum) as latent states, enabling traders to estimate real-time regime-shift probabilities and adapt allocations [2]. Filtering techniques also tackle microstructure noise in *illiquid markets*. Particle filters, for instance, reconstruct the latent "true" mid-price of sparsely traded assets (e.g., corporate bonds) from irregular, noisy transaction records [3]. As a one-dimensional demonstration, let us first write down the state-space model,

$$\begin{aligned} x_k &= f(x_{k-1}, w_k) \\ z_k &= h(x_k, u_k) \end{aligned} \tag{1}$$

where k denotes the time step, z_k is the observation, and x_k is the unobserved latent state. The noise terms w_k and u_k are mutually and temporally uncorrelated. The function f governs state evolution and h maps the state to observations. In practice we observe only z_k, which may have the same or lower dimension than x_k (the latter case is called partial observation and makes inference substantially harder due to information loss). Successful application of state-space models rests on two pillars:

1. A well-specified model with accurate parameters — traditionally this means understanding the functional forms of f and h and estimating their parameters reliably
2. An effective inference algorithm that estimate the hidden state x_k from the observation z_k — here, filtering algorithm.

In recent years neural networks have entered both parts of the equation. We can replace the conventional maps with learned versions: $f \to f_\theta$ and $h \to h_\theta$. Neural filtering algorithms have also emerged. We will first review the conventional numerical methods then discuss what are the possible ways for improvement leveraging neural networks.

1.1.1 The gist of performing filtering

Having a good model is only half the battle; we also need a reliable implementation. In research, novel filtering algorithms are developed and evaluated on synthetic data where the ground truth is known, allowing clear performance measurement. Given the state evolution f and observation model h, the goal is to estimate the hidden state x_k via the filtering distribution $p(x_k \mid z_{1:k})$. This is obtained recursively:

$$\begin{aligned} \textbf{Prediction:} \quad & p(x_k \mid z_{1:k-1}) = \int p(x_k \mid x_{k-1})\, p(x_{k-1} \mid z_{1:k-1})\, dx_{k-1} \\ \textbf{Update:} \quad & p(x_k \mid z_{1:k}) = \frac{p(z_k \mid x_k)\, p(x_k \mid z_{1:k-1})}{p(z_k \mid z_{1:k-1})} \end{aligned} \tag{2}$$

In financial applications (and most real-world settings), we do not aim to recover an exact deterministic value of x_k. Instead, the useful quantities are typically the mean m_k and covariance P_k of the filtering distribution. The filtering problem is generally a high-dimensional path integral and is intractable except in special cases (e.g., linear Gaussian SSMs). Different filtering methods (EKF, UKF, particle filters, etc.) use distinct approximations and mathematical frameworks. The key requirement for any successful filtering algorithm is that it produces numerically stable and accurate approximations to $p(x_k \mid z_{1:k})$ for known model dynamics.

1.1.2 Parameter estimation

Once a reliable filtering algorithm is available, it can be used to estimate the model parameters θ. We retain the functional forms of f and h but treat their parameters as variables to be learned. A good parameter set $\{\theta\}$ should assign high probability to the observed sequence. This is achieved by maximizing the marginal likelihood $p_\theta(z_{1:T})$, or equivalently minimizing the negative log-likelihood:

$$L(\theta) = -\sum_{k=1}^{T} \log\, p_\theta(z_k \mid z_{1:k-1}) \tag{3}$$

Since the logarithm is monotonically increasing, minimizing $L(\theta)$ maximizes the predictive density of each observation given past data [4].In a Bayesian setting, where parameters are given a prior $p(\theta)$, we instead minimize,

$$L(\theta) = -\sum_{k=1}^{T} \log\, p_\theta(z_k \mid z_{1:k-1}) - \log p(\theta), \tag{4}$$

which corresponds to maximum a posteriori (MAP) estimation.

1.2 The Kalman Family and semi-exact methods

For Linear-Gaussian systems, the Kalman Filter (KF) provides the optimal solution because the product of two Gaussian distribution stays as Gaussian. However, financial data rarely satisfies these constraints. Extensions such as the Extended Kalman Filter (EKF) and Unscented Kalman Filter (UKF) utilize linearization and sigma-point approximations, respectively. Either way, for Gaussian distribution, we only need the mean and variance at time k, m_k and P_k.

$$\begin{aligned} x_k =& f x_{k-1} + w_k, \quad w_k \sim \mathcal{N}(0, Q) \\ z_k =& h x_k + v_k, \quad v_k \sim \mathcal{N}(0, R) \end{aligned} \tag{5}$$

where a and h are constant in one dimension and matrices in higher dimension. When the kernel becomes non-linear, one essentially expand it around the last input.

$$\begin{aligned} f(x_{k-1}) &\approx \frac{\partial f(x)}{\partial x}\Big|_{x_{k-1}} \cdot x_{k-1} = f_k x_{k-1} \\ h(x_{k-1}) &\approx \frac{\partial h(x)}{\partial x}\Big|_{x_k} \cdot x_k = h_k x_k \end{aligned} \tag{6}$$

essentially introduce time dependence into the kernels but do not change the Gaussian assumption. This works fine if the higher order term are unimportant. UFK use sigma point to track the mean and variance and was able to capture some higher order correlations but it still assume Gaussian distribution. Appendix. A includes all the details such as key derivation. Kalman family mostly serves as a bench mark and have very limited application. Both EKF and UFK are methods to overcome the non-linearity in the problem, with more recent progress in this direction [5]. However, there are other difficulties as well, notably the curse of dimension. Attempts has been made to improve in this direction, notably Cubature Kalman Filter (CKF)[6] and Ensemble Kalman Filter (EnKF)[7, 8]. But here I do want to mention the latest progress on application of nerual network to kalman filter, known as KalmanNet[9, 10]. This is a method that replaces the Kalman Gain computation with a learned neural network while keeping everything else from the standard KF flow intact. The key is to train the RNN to make predictions about the Kalman gain without knowing what the noise distribution is. It gets the noise from the input data. While KalmanNet borrowed design from Kalman filter, it can be applied to non-linear, non-gaussian state space models.

1.3 Particle based methods

Particle based methods, also known as sequential Monte Carlo are the focus of this project. It is less limited compared to other approaches. There are tons of different particle-based methods, but the bottom line is how to represent a complicated distribution efficiently. Here we first introduce the concept of importance sampling. Here I want to explain why we choose to use system dynamics as the proposal instead of more general approach. Then set the things straight, we only use the system dynamics here.

As for particle weights, it should be corrected here that it is not an artifact of discretization alone. If we were to use particles, namely, samples to represent a distribution $p(x)$, we first need to discretize the phase space.

$$p(x) = \int dy\ p(y)\delta(y - x) \tag{7}$$

discretized version is,

$$p(x) \propto \sum_i \Delta p(x_i)\delta_{x,x_i} = \sum_i \omega_i \delta_{x,x_i} \tag{8}$$

where we have evenly sampled the phase space points and give them weights the realizes the actual distribution. To put this in the context of sequential Monte Carlo, we demonstrate the so-called Bootstrap particle filter (BPF), at step k, we have the realization of the distribution $\{x_i^{(k-1)}, \omega_i^{(k-1)}\} \sim p(x_{k-1}|z_{1:k-1})$ as input. In the evolution/prediction step,

$$\begin{aligned} &\text{For } i = 1 \text{ to } N: \\ &\qquad x_k^{(i)} \sim p(x_k \mid x_{k-1}^{(i)}) \\ &\qquad \tilde{w}_k^{(i)} = w_{k-1}^{(i)} \quad \text{(weights unchanged in basic prediction)} \end{aligned} \tag{9}$$

In the above step, the particle simply moved based on physics but their weight stays the same. But we need to use the observation data to score each update

$$\begin{aligned} &\text{For } i = 1 \text{ to } N: \\ &\qquad \tilde{w}_k^{(i)} = w_{k-1}^{(i)} \cdot p(z_k \mid x_k^{(i)}) \quad \text{(unnormalized importance weight)} \\ &\qquad w_k^{(i)} = \frac{\tilde{w}_k^{(i)}}{\sum_{j=1}^N \tilde{w}_k^{(j)}} \quad \text{(normalized weight)} \end{aligned} \tag{10}$$

The major problem with this simple approach is that $p(z_k|x^i)$ are most likely to be a small number. After few steps, most of weights will be so small due to multiplication of multiple small number that they becomes unimportant. The direct consequence is that effective sample size of the distribution becomes small and our discretized representation becomes even coarse grained. This problem is known as particle weight collapse or particle degeneracy. The most straightforward way to deal with this is resampling, which is to create an different realization for the same distribution with more balanced particle sample. Resample particles with replacement according to weights to avoid degeneracy (also called particle depletion or weight collapse). High-weight particles get duplicated; low-weight particles die out. This step is typically performed when $N_{\text{eff}} < N_{\text{threshold}}$ (e.g., $N_{\text{threshold}} = N/2$). The most straightforward resampling method is Multinomial resampling, namely, generate N independent samples from the categorical distribution:

$$\mathbf{x}_t^{(i)} \sim \text{Categorical}(\{w_t^{(j)}\}_{j=1}^N), \quad i = 1, \ldots, N \tag{11}$$

where particle $\mathbf{x}_t^{(j)}$ is selected with probability $w_t^{(j)}$. After resampling, set all weights to $w_t^{(i)} = 1/N$ for $i = 1, \ldots, N$. In the code, we will use systematic resampling, which is more efficient. For each sample we generate a single uniform random number $u \sim \mathcal{U}(0, 1/N)$ and define:

$$u^{(i)} = u + \frac{i-1}{N}, \quad i = 1, \ldots, N \tag{12}$$

Then select particle indices j_i such that:

$$\sum_{k=1}^{j_i - 1} w_t^{(k)} < u^{(i)} \leq \sum_{k=1}^{j_i} w_t^{(k)} \tag{13}$$

Set $\mathbf{x}_t^{(i)} = \mathbf{x}_t^{(j_i)}$ and $w_t^{(i)} = 1/N$ for all i. The particles relying on the duplicates or multiplicity to mimic the underlying distribution. As a consequence, we have sample impoverishment problem. Namely, to have more particles with non-trivial weights, we have to duplicate large weight particles while kill the small weight particles. After resampling, we set all particle having equal weight and continues the process.

Because all particles having equal weight, this means the weights carrying no information about the distribution and we rely on multiplicity to represent the distribution right after each resampling. This is extremely inefficient and hint that use multiplicity should've been the our first choice. Then the problem is very similar to numerical procedure to solve stochastic differential equations. Therefore, the prediction step stays the same because it doesn't touch the distribution, just change the basis, it is the update step that responsible to correct the particles. Because there is no diverse weight anymore, the only way to do this is to move the particles again. This is what particle flow particle filter does. These are observation guided evolution equations. When it comes to how, one has a good deal of freedom.

1.3.1 Flow and kernal flow

Alternatively, we don't use weights to represent the distribution at all, namely setting the weight to be uniform. In the update step, we are still using the dynamics to evolve the particle, but in the update step, instead, we move the particles again based on the observation so that the "corrected" distribution is a better realization of the target probability. We will discuss four different ways to move particles.

1. **Exact Flow:** Exact Daum-Huang (EDH) flow and Local Exact Daum-Huang (LEDH) flow[11, 12, 13] are using deterministic PDE to move the particle from prior to posterior. In practice, the algorithms EKF or UKF to move the filter forward. However, this is not the same as assuming the distribution is Gaussian. If the particles is sampled from non-Gaussian distribution, then after Gaussian update, the distribution will still be non-Gaussian. The upside of gaussian assumption is that we avoided computing score $\nabla \ln p$. But the downside is that strong nonlinear problem will break the flow.

2. **Stochastic Flow [14]** Conceptually, the most straightforward way to generalize ODE is SDE. We still make Gaussian assumption about the flow kernel but use stochastic differential equations to do the evolution. When it comes to stiff equations, the paper [15] proposed to use a differential equation to determine the optimal flow schedule instead of straightline.

3. **Invertible Particle flow-Particle filter:** Particle flow equations have discretizations errors, and they don't really transform the particle to the target probability. Think of the flow as a coordinate transformation, not only you should transform the map but also have Jacobian. PF-PF re-introduce the weights back into the picture and use the weight update to make adjustment to compensate the effect of this Jacobian. Because Jacobian is flow specific, if the flow equation kernel is identical across particles, in the case of EDH, then this Jacobian will not contribution and will be canceled by normalization. In the case of particle specific evolution, this Jacobian is highly non-trivial and will be important.

All above methods, on the implementation side, share one important feature. They use EKF/UKF to guide the flow. In appendix. B.2, we have shown how to use Gaussian approximation to derive a closed form solution to the evolution kernel but left out this important implementation details. All particles filters track the mean and empirical variances, and these are the real quantities that reflect the underlying state and uncertainty. Follow up works by different author might include their own custom adjustment. One confusing point I found is the interplay of empirical mean/covariance and EKF/UKF mean and covariance. To derive the evolution kernel, one don't need the parallel EKF to track the mean and variance at all, and this is what early implementation of EDH flow does. Later the authors started running a parallel EKF/UEK filter to track a less noisy mean and covariance. In [13], the author proposed the feedback mechanism to use the empirical mean but EKF covariance to determine the evolution kernel. At the end of the day, there is no one-fits-all solution. Most of the proposed fix are empirical adjustment. Given the underlying ground truth mean, one would like to stay close to the ground truth as possible, the gist of the fix is that one have to try to compare which mean is closer. Though, for highly non-linear system, empirical mean is most likely to be more reliable than EKF mean, given enough particle. In addition, the author of [16] claimed that they took their equation (10) and equation (11) from [13] for linearization performed at intermediate flow time. This is probably a typo, as I found no such equation in [13]. HMC for sampling in later follow up work by the author of [16].

1.3.2 kernel Based methods

kernel-embedded particle flow filter in an RK:[17, 18]: Instead of moving particles with certain schedule, we move the particle along the trajectory where the difference between the particle distribution with the target is decreasing as fast as possible. Define target posterior $\pi(x) := p(x_t|z_{1:t})$ and

intermediate density $q_\lambda(x)$ at pseudo-time $\lambda \in [0,1]$. With $q_0 = p(x_t|z_{1:t-1})$ (prior), $q_1 = \pi$ (posterior).

$$\frac{dx_\lambda}{d\lambda} = v_\lambda(x_\lambda) \tag{14}$$

Choose velocity v_λ to minimize KL divergence as fast as possible:

$$D_{KL}(q_\lambda \| \pi) = \int q_\lambda(x) \log \frac{q_\lambda(x)}{\pi(x)} dx \tag{15}$$

we need to formulate the above equation into an optimization problem. But the intuition of this method is clear, you want to evolve the point along a "straight" direction to the desire point. On the implementation side.

1.3.3 Optimal transport resampling

Previously, we have discussed the sample impoverishment problem introduced by low quality resampling. Therefore, we need to think about how to improve it. Before and after resampling, we are preserving the distribution or at latest trying to. We need to come up with the best way to do it. In[19], the author proposed to use Optimal transport to do the resampling. A easy way to see this is that, before the resampling, we have $\{\omega_i, x_i\}$, and after the resampling, we have $\{\tilde{\omega}_i, \tilde{x}_i\}$. If we think x_i are basis vector and ω_i are the coefficient, we are dealing with a linear transformation problem. The transformation will preserve the vector itself, or preserve the mean at least. One should not take this analogy too far. The key benefit is that OT resampling is deterministic instead of stochastic, this will avoid the sampling noise introduced by systematic resampling therefore removing its contribution to the variance of the particle, namely increasing the quality of covariance estimation. But this method limit the use of neural network architecture in the filtering algorithm. This method is not differentiable because it relies on solving a discrete Linear Programming (LP) problem to find the optimal transport plan. One just uses LP solvers like the Simplex algorithm or Interior Point methods. These algorithms don't need derivatives of the optimal solution with respect to inputs—they only need the cost matrix (particle distances) and constraints (matching weights). They work by walking along edges of the poly-tope or navigating through its interior. The key innovation of [20] is to introduce an entropy regularization so that the optimal coupling P^* is differentiable as function of the cost matrix therefore turning the whole resampling operation differentiable. The following is the loss function for the transport cost,

$$W^2_{2,\epsilon}(\alpha_N, \beta_N) = \min_{P \in S(a,b)} \sum_{i,j=1}^{N} p_{i,j} \left(c_{i,j} + \epsilon \log \frac{p_{i,j}}{a_i b_j} \right) \tag{16}$$

Another key innovation is that by adding the entropy term $\epsilon \log \frac{p_{i,j}}{a_i b_j}$ instead of $\epsilon \log p_{i,j}$, the author made it possible to determine the transport plan from the result of the dual problem, which is $\mathcal{O}(N)$, instead of $\mathcal{O}(N^2)$ like[19], which open the possibility for modeling more complicated system with large number of particles.

More recently, people also used neural network to solve the Monge problem itself [21].GradNetOT exploits Brenier's theorem, which says that under certain conditions (continuous densities, squared Euclidean cost), the Monge and Kantorovich problems are equivalent, and the unique transport map $T(x)$ is the gradient of a convex function $\phi(x)$, where $\phi(x)$ satisfies the Monge-Ampère equation

$$p(x) = q(\nabla \varphi(x)) \det(H_\phi(x)) \tag{17}$$

where $H_\phi = J_T$ is the $N \times N$ positive semi-definite Hessian of ϕ. But this architecture is not directly applicable to resampling because the transformation will push the continuous distribution $p(x)$ to the target distribution $q(y)$. We will discuss the possible way to make it work.

In addition, there are of course alternatives to OT resampling. Notable example is Particle Transformer[22, 23], where we replaces the OT resampler with a Transformer mechanism (self-attention) to interact particles with each other and re-weight them differentiably. To make this review more focused, we will not discuss it further. The key idea is that you demand the transformer to output particles with equal weights. Therefore, it learn from the data to put particles into the right positions.

Optimal transport resampling out-perform in certain scenarios but it's real power lies in that it makes it possible for entire filtering architecture differentiable. Because one can easily use neural network to model the dynamics or observation, which has been extensively discussed in [24]. I think there is no

point to make comprehensive review here as it is problem specific and require careful testing, there is no one-fits-all solution. This field is exploding and there are so many different architectures that one can try. As pointed out in the Bonus question 2, it will be nice that we don't have to solve the optimal transport problem repeatedly, because each resampling is a different set of initial particle and weights. One way to deal with this problem is amortized optimization[25], where you train a model to predict solutions to optimization problems rather than solving each problem from scratch. It's application to OT problem is known as Meta-OT [26]. The idea is actually very easy, when we run a filtering algorithm, we need to solve the resampling a lot of times. Each time, we will have a transport schedule, from the dual problem. We simply run filter a lot of times and record what kind of transport plan we will run into in such problems. Then we train neural network to make predictions.

1.4 Hamiltonian Monte Carlo

The whole point of using optimal transport, specifically its differentiable implementation, is to make sure the entire process is differentiable. To make our discussion more focused, our filter will be the invertible particle flow particle filter [16]. Without a differentiable resampling algorithm, we are forced to use Markov Chain Monte Carlo [27]. The idea is very simple, remember our goal is to find the best parameter set θ, such that negative log-likelihood is minimized. If the system is not fully differentiable, then we can use random walk to propose moves in the parameter space $\theta_{i+1} = \theta_i + w$, where w is a random vector in the parameter space. As a result, we will get a new loss function $L(\theta_{i+1})$, we simply compute the difference $L(\theta_{i+1}) - L(\theta_i) = \Delta L$, if $\Delta L < 0$, then we should accept the move. It is straightforward to incorporate prior distribution for θ. The question is what to do when $\Delta L > 0$. Simply rejecting all uphill moves would trap the chain at the nearest local minimum. Instead, the Metropolis-Hastings algorithm[28] accepts the proposed move with probability

$$\alpha = \min\big(1,\ e^{-\Delta L}\big) = \min\left(1,\ \frac{p(\theta_{i+1} \mid z_{1:T})}{p(\theta_i \mid z_{1:T})}\right) \tag{18}$$

where the second equality follows from $\Delta L = -\Delta \log p(\theta \mid z_{1:T})$. Even moves that increase the loss are accepted with probability $e^{-\Delta L}$, which allows the chain to escape shallow local optima. This acceptance rule guarantees detailed balance: if we define the transition probability from state θ_i to θ_j as $T(i \to j) = q(j|i)\,\alpha(i \to j)$, where q is the proposal distribution, then $p(\theta_i)T(i \to j) = p(\theta_j)T(j \to i)$. Detailed balance in turn guarantees that the Markov chain converges to the correct target distribution, which in our case is the posterior $p(\theta \mid z_{1:T})$[29].

The fundamental limitation of this random-walk Metropolis approach is that the proposal w is independent of the local geometry of the posterior. In a d-dimensional parameter space, a randomly chosen direction is almost surely nearly orthogonal to any special direction (such as the gradient or the principal axes of the posterior). To maintain a reasonable acceptance rate, the step size must shrink as $\mathcal{O}(d^{-1/2})$, which means the chain explores the posterior very slowly in high dimensions. This motivates using gradient information to construct informed proposals.

In the case of a fully differentiable pipeline, we can use Hamiltonian Monte Carlo[30, 29] to explore the parameter space. The idea is borrowed from classical mechanics. We introduce an auxiliary momentum variable $\mathbf{p}$ of the same dimension as θ and define the Hamiltonian as the total energy,

$$H(\mathbf{p}, \theta) = \underbrace{\frac{1}{2}\mathbf{p}^\top M^{-1}\mathbf{p}}_{\text{kinetic energy } K(\mathbf{p})} + \underbrace{U(\theta)}_{\text{potential energy}} \tag{19}$$

where $U(\theta) = -\log p(\theta \mid z_{1:T})$ is the negative log-posterior and M is a mass matrix (often set to the identity). The corresponding Hamilton's equations,

$$\frac{d\theta}{dt} = \frac{\partial H}{\partial \mathbf{p}} = M^{-1}\mathbf{p}, \qquad \frac{d\mathbf{p}}{dt} = -\frac{\partial H}{\partial \theta} = -\nabla_\theta U(\theta) \tag{20}$$

describe trajectories along which the total energy H is exactly conserved. In practice, we integrate these equations numerically using the *leapfrog* (Störmer-Verlet) integrator: half-step momentum update, full-step position update, half-step momentum update. The leapfrog scheme is symplectic, meaning it preserves phase-space volume and approximately conserves the Hamiltonian up to discretization error.

Each HMC iteration proceeds as follows. First, sample a fresh momentum $\mathbf{p} \sim \mathcal{N}(0, M)$. Then simulate L leapfrog steps starting from $(\theta_0, \mathbf{p}_0)$ to obtain a proposal $(\theta^*, \mathbf{p}^*)$. Finally, accept or reject

with the Metropolis criterion,

$$\alpha = \min\Big(1,\ e^{-(H(\mathbf{p}^*,\theta^*)-H(\mathbf{p}_0,\theta_0))}\Big) \tag{21}$$

Because the leapfrog integrator approximately conserves H, the energy difference $\Delta H \approx 0$, and the acceptance probability is close to 1 even for proposals that are far from the current point in parameter space. This is the key advantage over random-walk Metropolis: the momentum variable carries the state along the energy contour, allowing large, directed moves through parameter space that would be extremely unlikely under a random walk. The price we pay is that each step requires computing $\nabla_\theta U$, which means differentiating through the entire filtering pipeline.

1.4.1 Connection and difference with SGD

Both Hamiltonian Monte Carlo and stochastic gradient descent use the gradient $\nabla_\theta U(\theta)$, but for fundamentally different purposes. SGD performs point estimation: it seeks a single best parameter θ^* (MLE or MAP) by iterating $\theta \leftarrow \theta - \eta \nabla U$. Each step uses a different random seed for the particle filter, making the gradient stochastic, but SGD tolerates this because the noise averages out over hundreds of steps. SGD always moves in the gradient direction, there is no accept/reject step.

HMC, by contrast, performs posterior sampling: it characterizes the full distribution $p(\theta \mid z_{1:T})$, not just the mode. HMC requires a *fixed* random seed for the particle filter so that the energy landscape $U(\theta)$ is a deterministic function, because the leapfrog integrator simulates Hamiltonian dynamics and assumes the gradient is a smooth, deterministic force field. If the landscape changed randomly at each evaluation, the trajectory would see contradictory forces and diverge.

This leads to opposite gradient requirements. SGD needs gradients that are *correct on average* (unbiased), and can tolerate high variance. HMC needs gradients that are *smooth at every point*, and a single gradient cliff (where $|\nabla U|$ is enormous) can cause the leapfrog trajectory to explode. In the zero-temperature limit (infinitely sharp posterior concentrated at the mode), HMC's trajectory would trace out the same path as gradient descent. One can think of SGD as HMC without momentum coherence and without the acceptance correction that ensures the correct stationary distribution.

1.4.2 Implications for differentiable particle filters

When the filtering pipeline is fully differentiable (via soft or OT resampling), SGD or Adam can efficiently find the MAP point. When full posterior uncertainty is needed, HMC requires smooth gradients, and the choice of resampling strategy and filter type becomes critical. As we will see in Section 2.5, the tension between gradient quality (smoothness) and gradient information (low bias) is what drives the design of the differentiable particle filter pipeline.

1.5 Some further thoughts

2 Problems and experiments

2.1 State Space models

We first list all the models that I have used in the numerical experiments.

1. **Linear-Gaussian Model**

   Dynamics:
   $$X_n = F \cdot X_{n-1} + B \cdot V_n, \quad V_n \sim \mathcal{N}(0, I)$$

   Observation:
   $$Y_n = H \cdot X_n + D \cdot W_n, \quad W_n \sim \mathcal{N}(0, I)$$

   Parameters:

   - $F \in \mathbb{R}^{n_x \times n_x}$: State transition matrix
   - $B \in \mathbb{R}^{n_x \times n_v}$: Process noise matrix
   - $H \in \mathbb{R}^{n_y \times n_x}$: Observation matrix
   - $D \in \mathbb{R}^{n_y \times n_w}$: Observation noise matrix
   - $Q = BB^T$: Process noise covariance
   - $R = DD^T$: Observation noise covariance

   This is the simplest benchmark model for filtering.

2. **Acoustic Tracking Full Model (4 targets, 25 sensors)**

   Dynamics (4 independent targets):
   $$X_{t+1} = FX_t + w_t, \quad w_t \sim \mathcal{N}(0, V)$$
   where $F \in \mathbb{R}^{16\times 16}$ is block diagonal with 4 identical blocks, $X \in \mathbb{R}^{16}$

   Observation (additive amplitude):
   $$z_s = \sum_{i=1}^{4} \frac{\Psi}{r_{s,i}^2 + d_0} + \epsilon_s, \quad \epsilon_s \sim \mathcal{N}(0, \sigma_w^2)$$

   Parameters:

   - $V_{\text{true}} = \frac{1}{20}\text{diag}(V_{\text{single}}, \ldots, V_{\text{single}})$ (4 blocks)
   - $V_{\text{filter}} = \text{diag}(\begin{bmatrix} 3 & 0 & 0.1 & 0 \\ 0 & 3 & 0 & 0.1 \\ 0.1 & 0 & 0.03 & 0 \\ 0 & 0.1 & 0 & 0.03 \end{bmatrix}, \ldots)$ (4 blocks)
   - $\Psi = 10, d_0 = 0.1, \sigma_w = 0.1$
   - Sensors: 5×5 grid on $[0, 40] \times [0, 40]$. Basically evenly spread out in the whole area.

3. **Range-Bearing Model**

   Dynamics:
   $$X_t = FX_{t-1} + w_t, \quad w_t \sim \mathcal{N}(0, Q), \quad X = [x, y]^T$$

   Observation (nonlinear):
   $$\begin{bmatrix} \text{range} \\ \text{bearing} \end{bmatrix} = \begin{bmatrix} \sqrt{(x - x_s)^2 + (y - y_s)^2} \\ \arctan 2(y - y_s, x - x_s) \end{bmatrix} + \begin{bmatrix} v_r \\ v_b \end{bmatrix}$$
   where $v_r \sim \mathcal{N}(0, \sigma_r^2), v_b \sim \mathcal{N}(0, \sigma_b^2)$

   Parameters:

   - F: State transition (default: identity)

- Q: Process noise covariance (default: $0.01I_2$)
- (x_s, y_s): Sensor position (default: origin)
- $\sigma_r = 0.1$: Range noise std
- $\sigma_b = 0.01$: Bearing noise std (radians)

4. **Two-Sensor Bearing-Only Model**

   Dynamics (static):

   $$X_{t+1} = X_t, \quad X = [x, y]^T$$

   Observation:

   $$\begin{bmatrix}\text{bearing}_1 \\ \text{bearing}_2\end{bmatrix} = \begin{bmatrix}\arctan 2(y - y_{s,1}, x - x_{s,1}) \\ \arctan 2(y - y_{s,2}, x - x_{s,2})\end{bmatrix} + \begin{bmatrix}v_1 \\ v_2\end{bmatrix}$$

   where $v_i \sim \mathcal{N}(0, \sigma_b^2)$

   Parameters:

   - Prior: $\mu_0 = [3.0, 5.0]^T, \Sigma_0 = \begin{bmatrix}1000 & 0 \\ 0 & 2\end{bmatrix}$ (highly anisotropic)
   - Sensors: $[(3.5, 0), (-3.5, 0)]$
   - $\sigma_b = 0.2$: Bearing noise std
   - $R = \text{diag}(0.04, 0.04)$

5. **Stochastic Volatility Model**

   Dynamics (linear):

   $$x_t = \alpha x_{t-1} + \sigma w_t, \quad w_t \sim \mathcal{N}(0, 1)$$

   Observation (nonlinear, non-Gaussian):

   $$y_t = \beta \exp(x_t/2) v_t, \quad v_t \sim \mathcal{N}(0, 1)$$

   Parameters:

   - $\alpha = 0.91$: Persistence parameter
   - $\sigma = 1.0$: Volatility of volatility
   - $\beta = 0.5$: Scale parameter
   - Stationary variance: $\sigma^2/(1 - \alpha^2)$

6. **Lorenz 96 Model**

   Dynamics (chaotic, high-dimensional):

   $$\frac{dx_i}{dt} = (x_{i+1} - x_{i-2})x_{i-1} - x_i + F$$

   with cyclic boundary conditions, integrated using 4th-order Runge-Kutta

   Observation (linear, sparse):

   $$Y = HX + \epsilon, \quad \epsilon \sim \mathcal{N}(0, R)$$

   where H observes every 4th variable

   Parameters:

   - $n_x = 1000$: State dimension
   - $F = 8.0$: Forcing constant (chaotic regime)
   - $\Delta t = 0.05$: Integration time step
   - Observation interval: every 20 RK4 steps
   - $\sigma_{\text{obs}} = 1.0$: Observation noise std
   - $R = I_{n_y}$, where $n_y = 250$ (25% observed)

7. Kitagawa Model(Model from [27])

$$
\begin{aligned}
X_n &= \frac{X_{n-1}}{2} + 25\frac{X_{n-1}}{1+X_{n-1}^2} + 8\cos(1.2n) + V_n \\
Y_n &= \frac{X_n^2}{20} + W_n
\end{aligned}
\tag{22}
$$

where $X_1 \sim \mathcal{N}(0, 5)$ and $V_n \sim \mathcal{N}(0, \sigma_V^2)$
$W_n \sim \mathcal{N}(0, \sigma_W^2)$, where $\theta = (\sigma_v, \sigma_W)$ is our parameter. We only need to estimate the noise scale while the observation and dynamics are fixed.

2.2 Code organization and testing strategy

2.2.1 Configuration management

We use Hydra to manage the configuration with yaml files. Hydra is a framework that allows flexible management of the configurations. We start with a default base configuration for model and filters. In practice, we can override the default configuration with new configurations easily and the output file will automatically save a copy of the overwritten version of the configuration. In this way, we can easily create new experiments while keep track of the parameters corresponding to individual experiment outcome.

2.2.2 Code base organization

2.2.3 Testing

2.3 Part I

2.3.1 Kalman-Joseph update-condition number

The first topic we investigate is Joseph update and condition number. The following table is the calculation based on the 2-d linear Gaussian state space model with parameters:

$$F = \begin{pmatrix} 0.95 & 0 \\ 0 & 0.95 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$
$$H = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad D = \begin{pmatrix} 10^{-8} & 0 \\ 0 & 10^{-8} \end{pmatrix}$$
$$\mu_0 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad \Sigma_0 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

where the observation noise is of the order 10^{-8}, which led to $R \sim 10^-16$.

Table 1: Comparison of Kalman Filter with Joseph Form vs. Standard Update

Metric	**Joseph Form**	**Standard Update**
RMSE	9.708×10^{-9}	9.708×10^{-9}
Log-Likelihood	-566.840	-566.840
Performance Metrics		
Wall Time (s)	0.0203	0.0206
Time per Timestep (ms)	0.1016	0.1030
Peak Memory (MB)	0.274	0.273
Memory Increase (MB)	0.438	0.531
Numerical Stability		
Condition Number	1.0 (all timesteps)	∞ (all timesteps)

Small observation noise will lead to numerical instability and extremely large condition number in the updated covariance. In higher dimension, small noise is not the only such scenario but the argument for why Joseph form will restore numerical stability stays the same. Let us first take a look at what small R in 1-d looks like. With,

$$\lim_{R\to 0} K = \lim_{R\to 0} \frac{P^- h}{h^2 P^- + R} = \frac{1}{h} \tag{23}$$

With the standard update: $P^+ = (1 - Kh)P^- \approx 0$. But with Joseph form,

$$P^+ = (I - Kh)P^-(I - Kh) + K^2 R \sim \frac{R}{h^2} \tag{24}$$

we have see that $\frac{R}{h^2}$ is basically conditional number one because the noise scale is overall factor. This will legitimately restore numerical instability in higher dimension due to $k^2 R \to KRK^T$. Let $H = I$, $R = \epsilon I$, $P^- = I$. Then $K \approx I$. The condition number for the residue is one.

2.3.2 KF, EKF, UKF broken by strong nonlinear model

We first use the Stochastic Volatility model,

$$x_{t+1} = 0.91 x_t + w_t, \quad w_t \sim \mathcal{N}(0, 1) \tag{25}$$
$$y_t = 0.5 \exp(x_t/2) \cdot v_t, \quad v_t \sim \mathcal{N}(0, 1) \tag{26}$$

with the initial condition drawn from $N(0, \frac{\sigma^2}{1-\alpha^2})$, where $\alpha = 0.91, \sigma = 1$. The result shows that these two filter, though designed to handle non-linear problems, are completely broken by the Stochastic Volatility model. This is due to the fact that when trying to linearize the observation model, the noise term made sure that the mean is always zero. Even though we gave the filer an initial kick, it eventually converged to the zero mean. Another issue is that the variance received zero update due to zero h. In addition, I also did experiment on range bearing model,

State Space: $\mathbf{x}_t = [x_t, y_t]^\top \in \mathbb{R}^2$ (2D position)

State Evolution:

$$\mathbf{x}_t = F\mathbf{x}_{t-1} + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(0, Q) \tag{27}$$

where $F = I_2$ (constant position model) and $Q = 0.01 \cdot I_2$. The Observation Model is,

$$\mathbf{y}_t = h(\mathbf{x}_t) + \mathbf{v}_t, \quad \mathbf{v}_t \sim \mathcal{N}(0, R) \tag{28}$$

where the nonlinear observation function is:

$$h(\mathbf{x}_t) = \begin{bmatrix} \text{range}_t \\ \text{bearing}_t \end{bmatrix} = \begin{bmatrix} \sqrt{(x_t - x_s)^2 + (y_t - y_s)^2} \\ \arctan 2(y_t - y_s, x_t - x_s) \end{bmatrix} \tag{29}$$

Parameters:

- Sensor position: $\mathbf{x}_s = [x_s, y_s]^\top = [0, 0]^\top$
- Observation noise: $R = \text{diag}(\sigma^2_{\text{range}}, \sigma^2_{\text{bearing}})$
  - Range noise: $\sigma_{\text{range}} = 0.1$
  - Bearing noise: $\sigma_{\text{bearing}} = 0.01$ radians
- Initial state: $\mathbf{x}_0 \sim \mathcal{N}(\boldsymbol{\mu}_0, \Sigma_0)$ where $\boldsymbol{\mu}_0 = [1, 1]^\top$, $\Sigma_0 = I_2$

Jacobian: For the Extended Kalman Filter, the observation Jacobian is:

$$H(\mathbf{x}_t) = \frac{\partial h}{\partial \mathbf{x}}\bigg|_{\mathbf{x}_t} = \begin{bmatrix} \frac{x_t - x_s}{r_t} & \frac{y_t - y_s}{r_t} \\ -\frac{y_t - y_s}{r_t^2} & \frac{x_t - x_s}{r_t^2} \end{bmatrix} \tag{30}$$

where $r_t = \sqrt{(x_t - x_s)^2 + (y_t - y_s)^2}$ is the range.

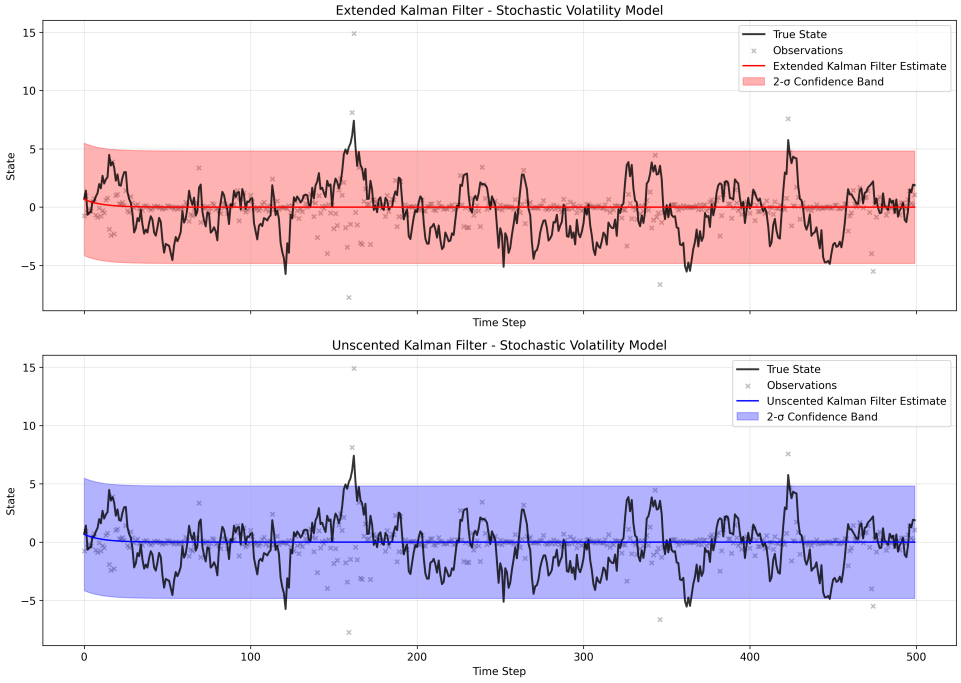

Figure 1: EKF and UKF result on stochastic volatility model

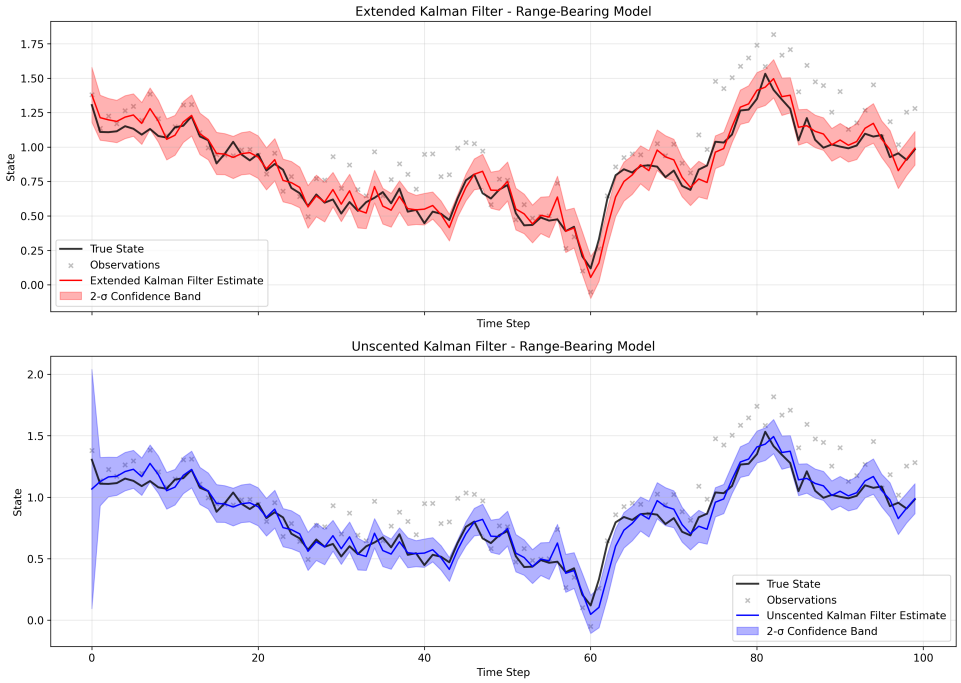

Figure 2: EKF and UKF result on range bearing model

we see that range bearing model works with UKF and EKF because we get can get reasonable Jacobian from the observation model.

2.3.3 Particle filter and particle degeneracy

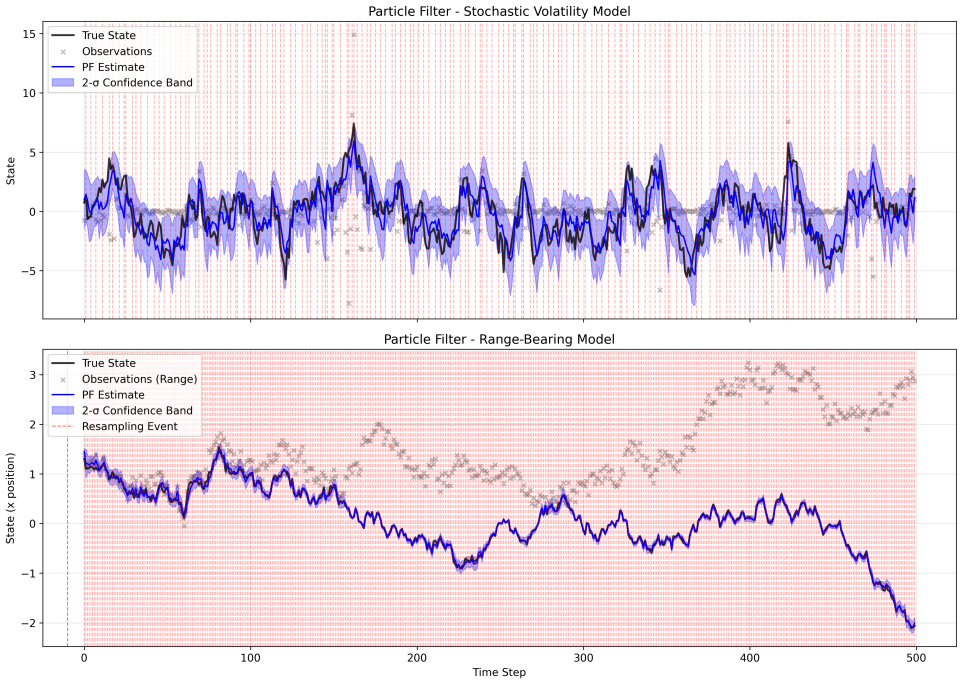

Figure 3: We see that particle filter tracking the means of both models pretty well

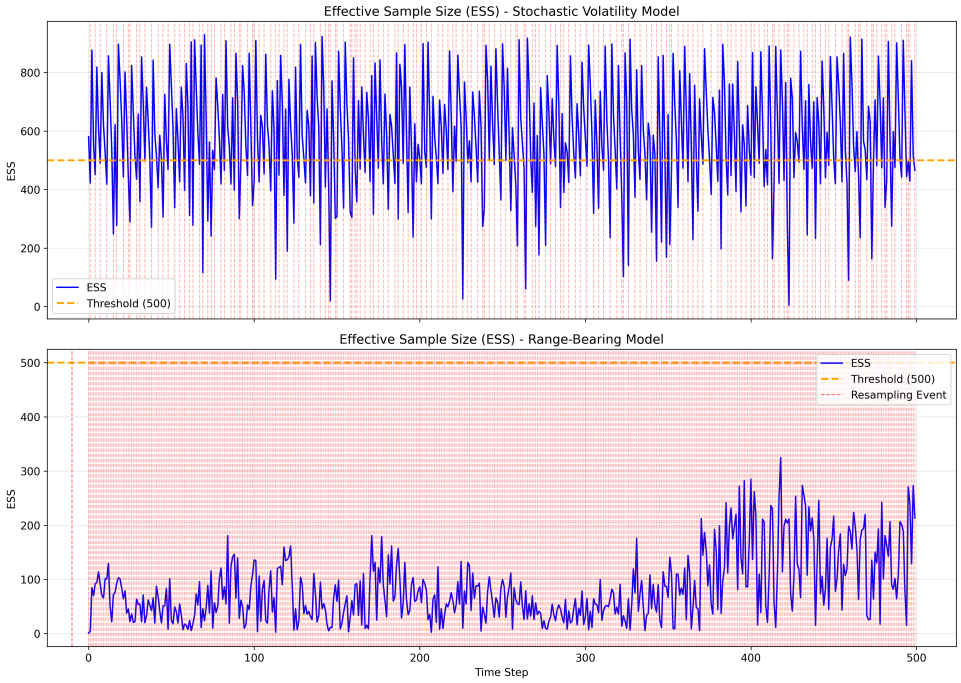

Figure 4: Effective sample size a function of time

The red lines are the times where we did resampling. It turns out that range bearing had very bad initialization but the particle filter somehow managed to increased sample size at later time. This is likely due to that particles were moved into the right position so even with very few particles, it can track the distribution very well. But a depleted sample size means that uncertainty estimation is unreliable.

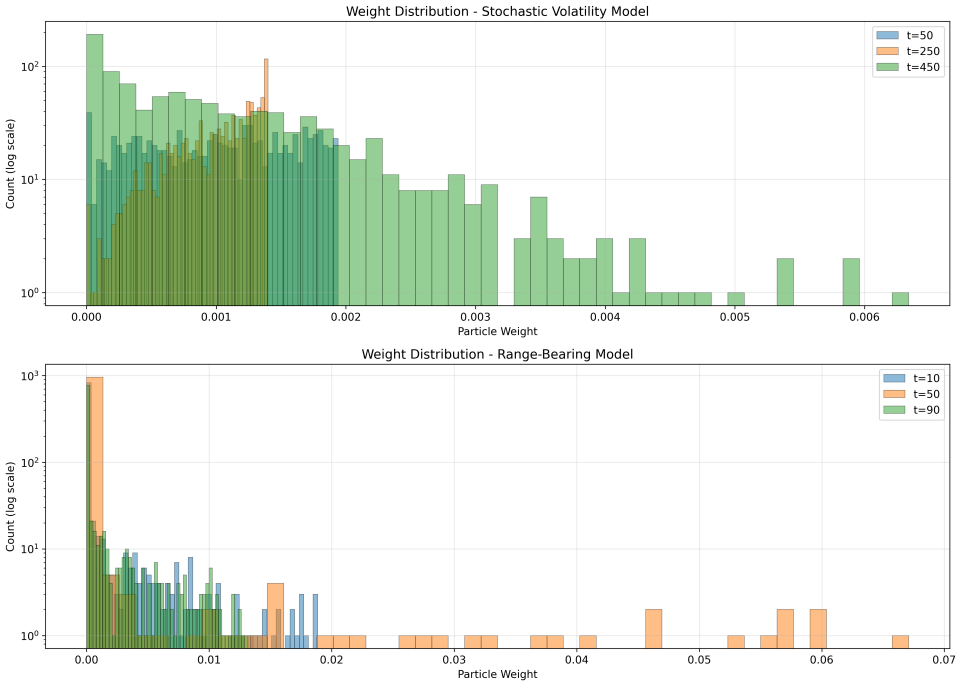

Figure 5: Weight distribution at three different time. It further demonstrate that bad initialization for range bearing model had collapsed weight distribution from the very beginning

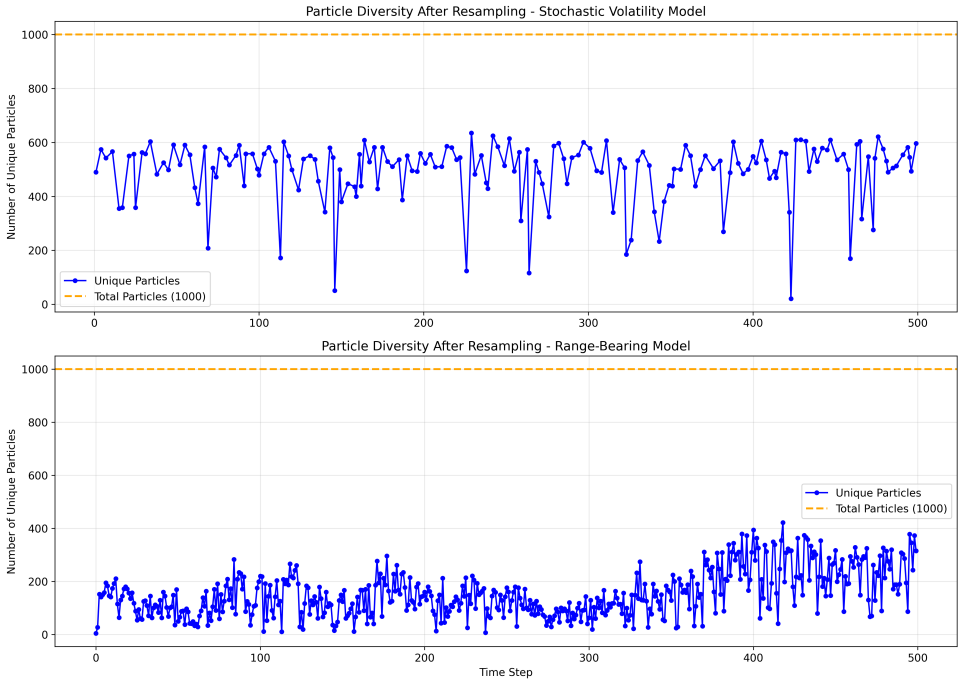

Figure 6: Particle diversity for the Stochastic Volatility model is reasonable.

2.3.4 EDH (LEDH) flow and filter

My flow filter and invertible flow particle filter did not work for acoustic tracking model or even simplified acoustic tracking model. I could not reproduce the result from [16]. But It works for range bearing model, so I include the performance for the range bearing model here. It is similar to the table 1 from [16]. The performance of PF is the baseline and all the flow and invertible filter will outperform Bootstrap particle filer. In addition, LEDH invertible has the best performance in terms of the accuracy but it is the slowest among all the filter.

Table 2: Particle Filter Comparison on Range-Bearing Model

Filter	**RMSE**	**Log-Lik**	**Wall Time (s)**	**Time/Step (ms)**	**Peak Mem (MB)**
PF	0.0610	745.41	50.81	101.63	16.05
EDH Flow	0.0593	—	7.26	72.55	1.48
EDH Invertible	0.0584	423.50	22.15	221.48	4.64
LEDH Flow	0.0584	—	228.09	2280.95	1.46
LEDH Invertible	0.0582	428.97	163.06	1630.56	1.67

Filter	**Particles**	**Resample Rate**	**Timesteps**
PF	1000	1.00	500
EDH Flow	500	—	100
EDH Invertible	2000	0.45	100
LEDH Flow	500	—	100
LEDH Invertible	500	0.37	100

2.3.5 Matrix kernel preventing collapse

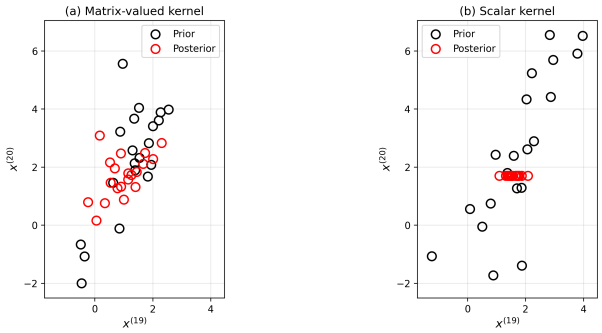

Figure 7: We can see clearly that by switching to matrix kernel, x_{20} no longer collapsed into one point

2.4 Part II

2.4.1 Stochastic flow

The stochastic particle flow filter of [14] generalises the deterministic EDH flow from an ODE to a stochastic differential equation:

$$d\mathbf{x} = \big[A(\lambda)\mathbf{x} + b(\lambda)\big]\, d\lambda + \sqrt{Q}\, dW_\lambda,$$

where $Q = \mathrm{diag}(q_1, \ldots, q_d)$ is a diagonal diffusion matrix. The drift includes a score correction (Corollary 2.1 of [14]) so that the posterior is exact in the continuous-time limit for any choice of Q.

Local correction for nonlinear observations. The drift parameters $A(\lambda)$ and $b(\lambda)$ are derived under the assumption of a linear observation model $h(\mathbf{x}) = H\mathbf{x}$. In the pure global formulation—where H is computed once at the prior mean $\bar{\eta}_0$ and the observation z is used directly—this linear model is self-consistent and no additional correction is needed.

However, in the re-linearisation framework (as in LEDH), one expands $h(\mathbf{x})$ at an intermediate point η during the flow. The first-order Taylor expansion

$$h(\mathbf{x}) \approx h(\eta) + H(\eta)(\mathbf{x} - \eta) = H(\eta)\mathbf{x} + \underbrace{\big[h(\eta) - H(\eta)\,\eta\big]}_{e},$$

naturally produces a zeroth-order residual $e = h(\eta) - H(\eta)\,\eta$ that accounts for the nonlinearity. In our local correction variant, we borrow this idea for the global formulation: compute $e_0 = h(\bar{\eta}_0) - H\bar{\eta}_0$ once, then replace z with $z_{\mathrm{corr}} = z - e_0$ in both the deterministic drift and the score correction. For linear observations $e_0 = 0$ and the two formulations coincide.

Empirically, this correction is essential for nonlinear models. On the range-bearing model, the uncorrected stochastic EDH diverges, while adding the local correction makes the filter work. The correction costs only one additional evaluation of $h(\bar{\eta}_0)$ per time step.

Stiffness and the λ schedule. The drift matrix $A(\lambda)$ has eigenvalues that are most extreme near $\lambda = 0$, where the precision $M(\lambda) = P_0^{-1} + \lambda H^\top R^{-1} H$ is dominated by the anisotropic prior P_0^{-1}. As $\lambda \to 1$, measurement information regularises M and the stiffness diminishes. With a sufficiently fine uniform grid (small $d\lambda$), the Euler–Maruyama integrator stays within the stability region even near $\lambda = 0$—a fine uniform schedule is not stressed by the stiff system.

In practice, one can do better with a geometric (exponential) schedule: step sizes $\varepsilon_j = \varepsilon_1 q^{j-1}$ with ratio $q = 1.2$, which concentrates steps near $\lambda = 0$ where they are most needed and uses larger steps near $\lambda = 1$ where the system is well-conditioned. This simple engineering choice already resolves the practical stiffness issue without solving any optimisation problem.

The companion paper [15] goes further, proposing an optimal homotopy schedule $\beta^*(\lambda)$ that minimises the nuclear-norm condition number of M via a boundary-value problem (BVP). We attempted to reproduce this but found both a sign error in the key formula and fundamental numerical issues.

Setup. Following [15] (Remark 3.2), define the precision matrix

$$M(\beta) = P_0^{-1} + \beta\, H^\top R^{-1} H,$$

with shorthands $J_{\mathrm{prior}} = P_0^{-1}$, $J_{\mathrm{meas}} = H^\top R^{-1} H$, and $\partial M/\partial\beta = J_{\mathrm{meas}}$. The nuclear-norm condition number is

$$\kappa_\nu(M) = \|M\|_\nu\, \|M^{-1}\|_\nu = \mathrm{tr}(M)\ \mathrm{tr}(M^{-1}),$$

and the optimal schedule solves the BVP (Theorem 3.1, Eqs 26–27):

$$\ddot{\beta} = \mu\, \frac{\partial \kappa_\nu}{\partial \beta}, \qquad \beta(0) = 0, \quad \beta(1) = 1.$$

Correct derivative of κ_ν. By the product rule and the matrix identity $dA^{-1}/d\theta = -A^{-1}(dA/d\theta)A^{-1}$ (Lemma A.1 of the paper):

$$\begin{aligned}\frac{\partial \kappa_\nu}{\partial \beta} &= \frac{\partial\, \mathrm{tr}(M)}{\partial \beta}\ \mathrm{tr}(M^{-1}) + \mathrm{tr}(M)\, \frac{\partial\, \mathrm{tr}(M^{-1})}{\partial \beta} \\ &= \mathrm{tr}(J_{\mathrm{meas}})\ \mathrm{tr}(M^{-1}) - \ \mathrm{tr}(M)\ \mathrm{tr}(M^{-1} J_{\mathrm{meas}}\, M^{-1}). \end{aligned} \tag{31}$$

The second term carries a **minus** sign. However, Eq (28) of [15], after translating $\nabla_x \nabla_x^\top \log h = -J_{\text{meas}}$, yields a **plus** sign instead.

Verification. We verify the sign in two independent ways.

Finite differences. Using central differences $\big(f(\beta+\varepsilon) - f(\beta-\varepsilon)\big)/(2\varepsilon)$ with $\varepsilon = 10^{-7}$ and the paper's numerical example (Section 4), the finite-difference values match the minus-sign formula to relative error $< 10^{-8}$ at every test point.

Table 3: Finite-difference verification of $\partial\kappa_\nu/\partial\beta$

β	Finite diff	Minus sign	Plus sign	FD matches
0.00	−559,000.44	−559,000.43	+561,729.57	minus
0.01	−3,762.72	−3,762.72	+3,991.69	minus
0.10	−41.83	−41.83	+71.20	minus
0.50	−0.31	−0.31	+9.59	minus
1.00	+0.59	+0.59	+5.55	minus

Isotropic sanity check. For $M = (c+\beta)I$ with $J_{\text{meas}} = I$, the condition number is $\kappa_\nu = n^2$ (constant), so $\partial\kappa_\nu/\partial\beta = 0$. The minus-sign formula gives $n/(c+\beta) - n/(c+\beta) = 0$, while the plus-sign formula gives $2n/(c+\beta) \neq 0$—incorrectly predicting a nonzero derivative for a constant function.

BVP analysis. Using the paper's parameters ($P_0 = \text{diag}(1000, 2)$, sensors at $(\pm 3.5, 0)$, $R = 0.04I$, $\mu = 0.2$):

- **Paper sign (+):** Both terms in the plus-sign formula are strictly positive (traces of PSD matrices with PD M), so $\ddot{\beta} > 0$ everywhere and β is strictly convex. A shooting scan over $\dot{\beta}(0) \in [-1, 2]$ gives $\beta(1) \geq 1.62$ for all initial velocities tested. The BVP has no solution.
- **Correct sign (−):** $|\partial\kappa_\nu/\partial\beta| \approx 559{,}000$ at $\beta = 0$ (caused by 500:1 anisotropy in P_0), making the raw BVP extremely stiff and intractable for shooting.

Log-κ fix. Replacing κ_ν with $\log \kappa_\nu$ in the objective gives

$$\ddot{\beta} = \mu \frac{\partial \log \kappa_\nu}{\partial \beta} = \mu \frac{1}{\kappa_\nu} \frac{\partial \kappa_\nu}{\partial \beta}.$$

At $\beta = 0$: $\kappa_\nu = 502$, so the driving term drops from 559,000 to $\approx 1{,}114$, making the BVP well-conditioned. Bisection shooting converges to $\dot{\beta}(0)^* = 1.6947$, yielding $\beta(1) = 1.0000000$.

The resulting schedule is nearly linear. The optimal schedule from the $\log \kappa_\nu$ BVP deviates from linear by at most $\max|\beta^* - \lambda| = 0.0195$. Compare with the paper's Figure 2(b), which shows $\max|\beta^* - \lambda| \approx 0.14$ (seven times larger). The condition-number reduction is correspondingly modest:

Table 4: Condition number comparison: linear vs. optimal schedule

λ	κ_ν(linear)	κ_ν(optimal)	ratio
0.01	43.3	31.6	0.73
0.05	11.1	9.4	0.85
0.10	6.9	6.3	0.91
0.50	4.0	4.0	1.00

This is not sufficient to reproduce the paper's Table 1 ($\text{tr}(P)$: 1535 → 1028, a 33% reduction).

Summary.

1. Eq (28) of [15] has a sign error: the second term should be minus, not plus.
2. The plus-sign BVP has no solution for the paper's own numerical example.

3. The correct minus-sign BVP is numerically intractable due to $O(10^5)$ derivatives near $\beta = 0$.

4. Replacing κ_ν with $\log \kappa_\nu$ makes the BVP solvable but yields a nearly linear schedule ($\max |\beta^* - \lambda| = 0.02$), insufficient to reproduce the paper's reported results.

5. The paper's Figure 2 and Table 1 show a much more aggressive schedule that cannot be obtained from the nuclear-norm condition number as described in Eq (28), with either sign. The authors likely used a different implementation than what the equation describes.

Our code (`stochastic_edh.py`) uses the $\log \kappa_\nu$ formulation with the correct minus sign, a Radau stiff solver, and grid-scan bracketing.

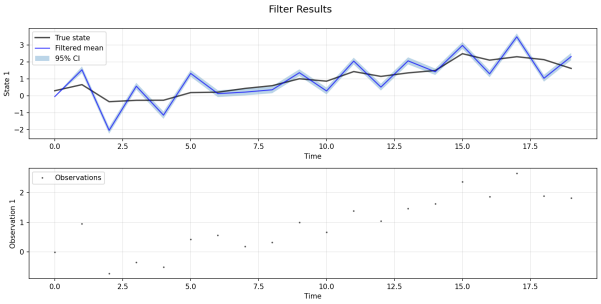

Figure 8: The current version of the SDE filter, when diffusion scale is set to 0.001

2.4.2 Soft resampling and OT resampling

Table 5: Resampling Method Comparison on Stochastic Volatility Model

Method	Param	RMSE	Log-Lik	Wall Time (s)	Time/Step (ms)	Resample Rate
Baseline						
PF (Systematic)	—	1.182	−436.29	3.65	7.31	0.356
Optimal Transport (OT) Resampling						
OT	$\epsilon = 0.1$	NaN	NaN	32.61	65.23	0.290
OT	$\epsilon = 0.3$	1.372	−477.29	25.04	50.08	0.390
OT	$\epsilon = 0.5$	1.379	−476.44	18.32	36.65	0.390
OT	$\epsilon = 1.0$	NaN	NaN	5.92	11.85	0.286
Soft Resampling						
Soft	$\alpha = 0.5$	1.173	−436.42	1.26	2.52	0.534
Soft	$\alpha = 0.7$	1.167	−436.04	1.25	2.51	0.420
Soft	$\alpha = 0.9$	1.175	−435.56	1.27	2.54	0.368

Here I only have the result on their face value, so I will not speculate why the result turn out this way.

2.5 Differentiable particle filter

The goal is to build a differentiable particle filter on kitagawa model,

$$\begin{aligned} X_n &= \frac{X_{n-1}}{2} + 25\frac{X_{n-1}}{1 + X_{n-1}^2} + 8\cos(1.2n) + V_n \\ Y_n &= \frac{X_n^2}{20} + W_n \end{aligned} \tag{32}$$

where $X_1 \sim \mathcal{N}(0, 5)$ and $V_n \sim \mathcal{N}(0, \sigma_V^2)$. $W_n \sim \mathcal{N}(0, \sigma_W^2)$, where $\theta = (\sigma_v, \sigma_W)$ is our parameter. One can clearly see that the key difficulty is the observation model, not because it is non-linear but it is bimodal. This means the observation is basically useless if we just plug in the filter we have used so far. Therefore, we take a three step approach.

1. We build a differentiable particle filter that works for easier model, this is mostly test the pipeline of differentiable particle filter
2. we reproduce the PMCMC paper
3. we improve the ledh invertible particle flow particle filter and plug it into the dpf pipeline.

We now discuss each step in detail. Step 1 requires building the differentiable pipeline itself and choosing between optimization (SGD) and sampling (HMC). Step 2 provides a gradient-free benchmark via PMCMC[27]. Step 3 introduces Particle Gibbs to overcome the limitations discovered in steps 1 and 2.

2.5.1 SGD-based parameter estimation (Step 1)

The simplest way to test the differentiable particle filter pipeline is to use gradient-based optimization. Given the differentiable pipeline, we treat the negative log-posterior as a loss function and minimize it with Adam or SGD,

$$\theta \leftarrow \theta - \eta \nabla_\theta [-\log p(z_{1:T} \mid \theta) - \log p(\theta)] \tag{33}$$

where $\log p(z_{1:T}|\theta)$ is the log marginal likelihood from the particle filter. Each optimization step uses a different random seed for the particle filter, making the gradient stochastic, but SGD handles this naturally by averaging over hundreds of steps. This is the approach of PF-net[31], which introduced soft resampling specifically to give nonzero gradient through the resampling step. Corenflos et al.[20] later used OT resampling for the same purpose.

SGD-based estimation finds the MAP point efficiently but provides no uncertainty quantification, it returns a single best estimate θ^* without characterizing how confident we should be in that estimate. We start with easier models (linear-Gaussian) to verify the pipeline works, then move to Kitagawa.

Add MAP estimation results: convergence curves for linear-Gaussian and Kitagawa models, compare BPF vs LEDH.

2.5.2 HMC through the particle filter (Step 1)

For full posterior estimation (uncertainty quantification), we use HMC on a deterministic likelihood surface by fixing the particle filter seed. This gives us the posterior distribution $p(\theta \mid z_{1:T})$, not just the mode. However, the particle filter's log-likelihood surface has features that make HMC challenging.

The differentiability-bias trade-off. A common claim is that systematic resampling is "not differentiable" and therefore cannot be used in differentiable particle filters. This is misleading. Systematic resampling consists of computing cumulative weights, generating uniform points, finding indices via searchsorted, and gathering particles. The searchsorted step maps continuous weights to integer indices, a piecewise-constant function, so

$$\frac{\partial I_k}{\partial w_j} = 0 \quad \text{almost everywhere}$$

The code runs fine; TensorFlow computes a gradient through this, it is just zero. The gradient carries no information about how the resampling depends on weights, but there is no crash or error. Using `stop_gradient` explicitly cuts all gradient through resampling, the effect is almost identical to the natural zero-gradient but more controlled.

Soft resampling replaces the discrete index selection with a continuous relaxation. The weights are blended with a uniform distribution,

$$\tilde{w}_i = \alpha\, w_i + (1 - \alpha)/N \tag{34}$$

so that $\partial \tilde{w}_i / \partial w_i = \alpha \neq 0$. OT Sinkhorn produces a differentiable transport plan T_{ij} so that $x'_i = \sum_j T_{ij} x_j$. Both approaches give nonzero gradients through resampling, but at a cost.

The critical distinction is that HMC is not SGD. SGD averages over many noisy gradient steps and tolerates high variance. HMC's leapfrog integrator simulates Hamiltonian dynamics and assumes the gradient is a smooth, deterministic force field. With `stop_gradient`, the gradient is biased but smooth, giving a valid (if slightly wrong) Hamiltonian for the leapfrog to simulate. With soft resampling, the gradient through resampling of N particles has high variance, causing leapfrog trajectories to accumulate noise at each step and diverge into extreme parameter regions. Therefore, soft or OT resampling is preferred for SGD (less bias), while systematic resampling with `stop_gradient` is more stable for HMC (smoother energy surface).

The gradient cliff problem. The observation log-likelihood for Gaussian noise is

$$\log p(y \mid x, \sigma) = -\frac{1}{2}\log(2\pi\sigma^2) - \frac{(y - h(x))^2}{2\sigma^2} \tag{35}$$

As $\sigma \to 0$, the $-\log\sigma$ term increases while the $(y - h(x))^2/\sigma^2$ term explodes. There is a narrow ridge where these balance, creating a sharp cliff in the log-likelihood surface. The gradient at the cliff edge is enormous, far too large for the leapfrog step size to handle. LEDH amplifies this problem: its 29-step Jacobian chain $\prod_{j=1}^{29} \det(I + \Delta\lambda\, A_j(\theta))$ is extremely sensitive to θ, creating additional curvature that causes leapfrog divergence.

Add experimental results: filter × resampling comparison table (acceptance rates, convergence, time/step). Currently BPF+systematic is the only working config for direct HMC.

This limitation, that only the simplest filter (BPF with systematic resampling) works with direct HMC, motivates steps 2 and 3.

2.5.3 PMMH: gradient-free benchmark (Step 2)

The Particle Marginal Metropolis-Hastings (PMMH) algorithm from [27] takes a completely different approach: it replaces the intractable likelihood $p(z_{1:T}|\theta)$ with the particle filter's unbiased estimate $\hat{p}(z_{1:T}|\theta)$, then uses standard Metropolis-Hastings. At each iteration, we propose θ^* via random walk, run a particle filter with a fresh random seed to obtain $\hat{p}(z_{1:T}|\theta^*)$, and accept with probability

$$\alpha = \min\left(1,\ \frac{\hat{p}(z_{1:T}|\theta^*)\, p(\theta^*)}{\hat{p}(z_{1:T}|\theta^{(i)})\, p(\theta^{(i)})}\right) \tag{36}$$

The remarkable result (their Theorem 2) is that this leaves the correct posterior invariant for *any* $N \geq 1$ particles. No gradients are needed, so gradient cliffs and resampling discontinuities are completely irrelevant.

In their Section 3.1, they apply PMMH to exactly our Kitagawa model with $\sigma_V^2 = 10$, $\sigma_W^2 = 1$, using $N = 5000$ particles, $T = 500$ time steps, and 50,000 MCMC iterations with Inverse Gamma priors on the variances. The acceptance rate depends strongly on N: roughly 10% at $N = 100$, 50% at $N = 500$, and 80% at $N = 2000$ (for $T = 100$). This is consistent with their Theorem 1, which shows that the variance of the likelihood estimate scales as $\mathcal{O}(T/N)$, doubling T requires doubling N to maintain the same acceptance rate.

An important warning from their paper (p. 283): standard block-wise MCMC (updating latent states $x_{1:T}$ one block at a time, then updating θ) "tends to become trapped in a local mode of the multimodal posterior... and results in an overestimation of the true value of σ_V." PMMH and Particle Gibbs did not have this trapping problem, suggesting that the difficulty we face with the Kitagawa model is partly a multimodality issue, not just a gradient cliff issue. The trade-off is that each PMMH step is cheap (one particle filter run, no gradient computation) but mixing is slow because the proposals are random walk.

Add our PMMH reproduction results on Kitagawa model: posterior histograms, acceptance rates, comparison with their Fig. 3.

2.5.4 Particle Gibbs: decoupling parameters and states (Step 3)

Steps 1 and 2 reveal three core problems: gradient cliffs when differentiating through the particle filter (Step 1), weight collapse from LEDH's accumulated Jacobian products (Step 1), and bimodality trapping for the Kitagawa observation model (Step 2). The key insight, also from [27], is that instead of marginalizing out the latent states $x_{0:T}$ (which requires differentiating through the particle filter), we can target the *joint* posterior $p(\theta, x_{0:T} \mid z_{1:T})$ via Gibbs sampling, alternating between two steps.

θ-step (HMC on a smooth surface). Given a fixed trajectory $x_{0:T}$, the conditional posterior of θ is

$$\log p(\theta \mid x_{0:T}, z_{1:T}) = \sum_{t=1}^{T} \log p(x_t \mid x_{t-1}, \theta) + \sum_{t=1}^{T} \log p(z_t \mid x_t, \theta) + \log p(\theta) + \text{const} \tag{37}$$

Each term is a Gaussian log-density. No particle filter is involved, there is no resampling, no Jacobian accumulation, no weight collapse. For the Kitagawa model with $\theta = (\sigma_V, \sigma_W)$, this becomes

$$\log p(\sigma_V, \sigma_W \mid x_{0:T}, z_{1:T}) = -T\log\sigma_V - \frac{1}{2\sigma_V^2}\sum_{t=1}^{T}(x_t - f(x_{t-1}, t))^2 - T\log\sigma_W - \frac{1}{2\sigma_W^2}\sum_{t=1}^{T}\left(z_t - \frac{x_t^2}{20}\right)^2 + \log p(\sigma_V, \sigma_W) \tag{38}$$

where $f(x,t) = x/2 + 25x/(1+x^2) + 8\cos(1.2t)$. This is a smooth, cheap $\mathcal{O}(T)$ computation with exact gradients. HMC can use large step sizes and achieve 60–80% acceptance on this surface, compared to 0–10% when differentiating through the particle filter.

x-step (Conditional SMC). We sample a new trajectory $x_{0:T} \sim p(x_{0:T} \mid \theta, z_{1:T})$ using a particle filter with fixed θ. No gradients are needed, this is pure forward simulation. The standard approach is Conditional Sequential Monte Carlo (CSMC)[27, 32], which pins one particle (the *reference*) to the previous iteration's trajectory $x^*_{0:T}$ while letting $N-1$ free particles explore. At each time step, the reference particle follows the prescribed path deterministically, while free particles are propagated and weighted normally. At the final time step, a trajectory is sampled from the weighted particles and becomes the reference for the next Gibbs iteration.

Reference pinning serves two purposes. First, it guarantees ergodicity, the chain can always "stay put" by selecting the reference trajectory. Second, for bimodal models like Kitagawa where $y = x^2/20$ admits both $+\sqrt{20y}$ and $-\sqrt{20y}$, the free particles can explore both modes while the reference anchors the sampler. Ancestor sampling[32] further improves mixing by allowing the reference particle's history to "detach" and graft onto any other particle's past, dramatically reducing path degeneracy.

The Gibbs decomposition solves all three problems identified in Steps 1 and 2:

- Gradient cliffs from Step 1 are eliminated, the θ-step is closed-form.
- Weight collapse from LEDH is eliminated, no Jacobian accumulation in the θ-step.
- Bimodality trapping from Step 2 is addressed by CSMC reference pinning in the x-step.

The implementation proceeds in two phases. Phase 1 uses a bootstrap particle filter for the CSMC x-step, which is the simplest and validates the framework. Phase 2 replaces it with LEDH CSMC, where the flow is applied only to the $N-1$ free particles while the reference stays pinned, providing better proposals for challenging high-dimensional models. This is the "improve LEDH and plug into DPF" from the three-step plan above.

Add PGibbs + HMC results: θ-step acceptance rate on closed-form surface, CSMC trajectory diversity, comparison with direct HMC through PF and PMMH.

2.5.5 Additional notes and practical considerations

Leapfrog divergence. The mechanism by which direct HMC fails is instructive. The leapfrog trajectory starts in a reasonable region, passes through the likelihood cliff near small noise values (where the log-density changes sign), and the enormous gradient launches the state into an extreme parameter region. The Metropolis acceptance correctly rejects such diverged proposals (with acceptance probability $e^{-\Delta H} \approx 0$), but the chain makes no progress. Two remedies exist: gradient clipping, which caps $|\nabla U|$ at a threshold C (a practical hack that preserves detailed balance because the true energy is still used for acceptance), and NUTS (No U-Turn Sampler), which adaptively truncates the trajectory when it starts doubling back[29]. Both are worth trying for particle filter likelihoods.

Seed handling. HMC uses a *fixed* seed for the particle filter, making the likelihood surface deterministic for gradient computation. PMMH uses a *fresh* seed each evaluation, the stochasticity is part of the algorithm, not a bug. On rejection, PMMH keeps the old likelihood estimate rather than re-evaluating with new randomness.

Parameterization. The PMCMC paper uses variances σ^2 with Inverse Gamma priors (conjugate for Gaussian variance), while our setup uses standard deviations σ with an exponential bijector (log-space HMC). The gradient of $\log p(z|\sigma)$ with respect to $\log\sigma$ involves an extra chain rule factor of σ, which amplifies gradient magnitudes for large σ. Switching to a σ^2 parameterization might smooth the landscape.

Likelihood variance. From Theorem 1 of [27], the variance of the particle filter's likelihood estimate scales as $\mathrm{Var}(\hat{Z}^N/Z) \leq C \cdot T/N$. Doubling the number of time steps T requires doubling the number of particles N to maintain the same MCMC acceptance rate. For $T = 100$, $N = 1000$ should suffice; for $T = 500$, they needed $N = 5000$.

Organize these notes further as needed. Some may be moved to appendix or removed.

A Kalman Family

Notation:

- $\mathbf{m}_t^-$= *a priori* state estimate (mean of the predicted distribution)
- $\mathbf{m}_t$= *a posteriori* state estimate (mean of the updated distribution)
- $\mathbf{P}_t^-$ = *a priori* error covariance
- $\mathbf{P}_t$ = *a posteriori* error covariance
- $p(x_k|z_{1:k-1})$= prior distribution
- $p(x_k|z_{1:k})$ = posterior Distribution

we use the following initial condition,

$$p(x_0) = \mathcal{N}(x_0|m_0, P_0) \tag{39}$$

For Kalman filter, the noise and operators are both Gaussian, so the distributions stay at Gaussian at all times. All we need to do is to keep track of the mean and variance of the prior distribution $p(x_k|y_{1:k})$. In the predict step, the goal is to compute $p(x_t|z_{1:t-1})$ from $p(x_{t-1}|z_{1:t-1})$. Given $p(x_{t-1}|z_{1:t-1}) = \mathcal{N}(x_{t-1}|m_{t-1}, P_{t-1})$, we have:

$$p(x_t|z_{1:t-1}) = \int dx_{t-1}\, p(x_t|x_{t-1})\, p(x_{t-1}|z_{1:t-1}) \tag{40}$$

From the state evolution model:

$$p(x_t|x_{t-1}) = \mathcal{N}(x_t|fx_{t-1}, Q) \tag{41}$$

predicted/evolved mean is,

$$\begin{aligned} m_t^- &= \mathbb{E}[x_t|z_{1:t-1}] && (42)\\ &= fm_{t-1} && (43) \end{aligned}$$

predicted/evolved variance

$$\begin{aligned} P_t^- &= \mathrm{Var}[x_t|y_{1:t-1}] && (44)\\ &= Q + f^2 P_{t-1} && (45) \end{aligned}$$

from the predict step, we get,

$$p(x_t|z_{1:t-1}) = \mathcal{N}(x_t|m_t^-, P_t^-) \tag{46}$$

where

$$\boxed{m_t^- = am_{t-1}, \quad P_t^- = a^2 P_{t-1} + Q} \tag{47}$$

In the update step, we need to incorporate observation z_t using Bayes' rule.

$$p(x_t|z_{1:t}) = \frac{p(z_t|x_t)\, p(x_t|z_{1:t-1})}{p(z_t|z_{1:t-1})} \tag{48}$$

From the observation model:

$$p(z_t|x_t) = \mathcal{N}(z_t|hx_t, R) \tag{49}$$

We need to compute the product of two Gaussians:

$$\begin{aligned} p(x_t|z_{1:t}) &\propto \mathcal{N}(z_t|hx_t, R) \cdot \mathcal{N}(x_t|m_t^-, P_t^-) && (50)\\ &\propto \exp\left(-\frac{1}{2}\frac{(z_t - hx_t)^2}{R}\right) \exp\left(-\frac{1}{2}\frac{(x_t - m_t^-)^2}{P_t^-}\right) && (51) \end{aligned}$$

Expanding the exponents:

$$-\frac{1}{2}\left[\frac{(z_t - hx_t)^2}{R} + \frac{(x_t - m_t^-)^2}{P_t^-}\right] = -\frac{1}{2}\left[\frac{z_t^2 - 2z_t hx_t + h^2 x_t^2}{R} + \frac{x_t^2 - 2x_t m_t^- + (m_t^-)^2}{P_t^-}\right] \tag{52}$$

Collecting terms in x_t^2 and x_t:$-\frac{1}{2}\left[\left(\frac{h^2}{R}+\frac{1}{P_t^-}\right)x_t^2-2\left(\frac{hz_t}{R}+\frac{m_t^-}{P_t^-}\right)x_t+\text{const}\right]$. This is a Gaussian in x_t. Completing the square, we have $\frac{1}{P_t}=\frac{h^2}{R}+\frac{1}{P_t^-}$. P is the uncertainty so the inverse of P is like precision, by adding information about the measurement $\frac{h^2}{R}$, we get additional precision which is exactly what we are aiming for. When $\frac{h^2}{R}=0$, namely large noise from observation, we say we can't trust the measurement and we should stick with the dynamics. We see that both mean and variance will receive no update. On the other hand, if $\frac{h^2}{R}\ll\frac{1}{P_t^-}$, then observation will carry more weight and the result from dynamics should be disregarded. The updated variance is

$$P_t=\frac{RP_t^-}{h^2P_t^-+R} \tag{53}$$

For simplicity, we define the **Kalman gain:**

$$K_t=\frac{P_t^-h}{h^2P_t^-+R} \tag{54}$$

Then:

$$P_t=(1-K_th)P_t^- \tag{55}$$

After some algebra:

$$m_t=m_t^-+K_t(z_t-hm_t^-) \tag{56}$$

where we denote the innovation $v_t=z_t-hm^-$, namely the different between the real observation with the predicted observation from prior mean. In summary, we get,

$$\boxed{\begin{aligned}K_t&=\frac{P_t^-h}{h^2P_t^-+R}\\ v_t&=y_t-hm_t^-\\ m_t&=m_t^-+K_tv_t\\ P_t&=(1-K_th)P_t^-\end{aligned}} \tag{57}$$

at this point, we can already see that what filter does is to correct the distribution based on the observed data point. Essentially, we want the mean and covariance from the updated distribution at each time step. For higher dimension, all we have to do is to promote the scalar recursive relation into matrices.
Prediction Step:

$$\hat{\mathbf{x}}_t^-=\hat{\mathbf{F}}\hat{\mathbf{x}}_{t-1}^+ \tag{58}$$

$$\mathbf{P}_t^-=\hat{\mathbf{F}}\mathbf{P}_{t-1}^+\hat{\mathbf{F}}^T+\mathbf{Q} \tag{59}$$

Correction Step:

$$\hat{\mathbf{K}}_t=\mathbf{P}_t^-\mathbf{H}^T\left[\mathbf{H}\mathbf{P}_t^-\mathbf{H}^T+\mathbf{R}\right]^{-1}\quad\text{(Kalman gain)} \tag{60}$$

$$\hat{\mathbf{x}}_t^+=\hat{\mathbf{x}}_t^-+\mathbf{K}_t(\mathbf{z}_t-\mathbf{H}\hat{\mathbf{x}}_t^-) \tag{61}$$

$$\hat{\mathbf{P}}_t^+=(\mathbf{I}-\mathbf{K}_t\mathbf{H})\mathbf{P}_t^- \tag{62}$$

We know that Kalman filter requires linear models and Gaussian noise. But this is very restrictive. For the cases where the noises are Gaussian but models are non-linear, we have extended Kalman filter (EKF) and unscented Kalman Filter (UKF). The idea for the extension is somehow make approximation so that we can use Kalman filter. EKF linearizes the models via first-order Taylor expansion around current estimate.

$$\mathbf{F}_t=\left.\frac{\partial f}{\partial\mathbf{x}}\right|_{\mathbf{x}=\hat{\mathbf{x}}_{t-1}^+},\quad\mathbf{H}_t=\left.\frac{\partial h}{\partial\mathbf{x}}\right|_{\mathbf{x}=\hat{\mathbf{x}}_t^-} \tag{63}$$

Apply Kalman filter equations using time-varying $\mathbf{F}_t,\mathbf{H}_t$. Evolve using time-dependent operators,

$$\hat{\mathbf{x}}_t^-=f(\hat{\mathbf{x}}_{t-1}^+) \tag{64}$$

$$\mathbf{P}_t^-=\mathbf{F}_t\mathbf{P}_{t-1}^+\mathbf{F}_t^T+\mathbf{Q} \tag{65}$$

Update using $\mathbf{H}_t$ and predicted measurement $\hat{\mathbf{z}}_t^- = h(\hat{\mathbf{x}}_t^-)$.

$$\hat{\mathbf{K}}_t = \mathbf{P}_t^- \mathbf{H}_t^T \left[\mathbf{H}_t \mathbf{P}_t^- \mathbf{H}_t^T + \mathbf{R}\right]^{-1} \quad \text{(Kalman gain)} \tag{66}$$

$$\hat{\mathbf{x}}_t^+ = \hat{\mathbf{x}}_t^- + \mathbf{K}_t(\mathbf{z}_t - \mathbf{H}_t \hat{\mathbf{x}}_t^-) \tag{67}$$

$$\hat{\mathbf{P}}_t^+ = (\mathbf{I} - \mathbf{K}_t \mathbf{H}_t)\mathbf{P}_t^- \tag{68}$$

However, there is strong limitations of this approach, namely, only first-order accurate; can diverge for strong nonlinearity; requires Jacobian computation (may be expensive or analytically intractable); assumes Gaussian noises. As for the UKF, propagate uncertainty through nonlinear functions using **sigma points** (also called unscented points) - deterministically chosen samples that capture mean and covariance. Here we further introduce some definition:

- $\mathbf{P}_{zz,t}^-$ = predicted measurement covariance
- $\mathbf{P}_{xz,t}^-$ = cross-covariance between state and measurement

This is because Kalman gain has a more general form

$$\mathbf{K}_t = \mathbf{P}_{xz,t}^- \big(\mathbf{P}_{zz,t}^-\big)^{-1} \tag{69}$$

where

- $\mathbf{P}_{xz,t}^- = \mathrm{Cov}(\mathbf{x}_t - \hat{\mathbf{x}}_t^-, \mathbf{z}_t - \hat{\mathbf{z}}_t^-)$ (state–innovation cross-covariance)
- $\mathbf{P}_{zz,t}^- = \mathrm{Cov}(\mathbf{z}_t - \hat{\mathbf{z}}_t^-)$ (innovation covariance)

This expression is exact for the linear-Gaussian case and approximate (but very effective) for nonlinear cases when the covariances are estimated via linearization (EKF) or sigma points (UKF). Associated with the sigma points are two sets of weights used for computing the mean and covariance after nonlinear transformation:

$$W_0^{(m)} = \frac{\lambda}{n+\lambda} \tag{70}$$

$$W_0^{(c)} = \frac{\lambda}{n+\lambda} + (1 - \alpha^2 + \beta) \tag{71}$$

$$W_i^{(m)} = W_i^{(c)} = \frac{1}{2(n+\lambda)}, \quad i = 1, \ldots, 2n \tag{72}$$

where

- $W_i^{(m)}$ are used to recover the transformed **mean**,
- $W_i^{(c)}$ are used to recover the transformed **covariance**,
- β is typically set to 2 (optimal for Gaussian distributions).
- α - Spread Control Parameter, we set it to 10^{-3}
- κ - Secondary Scaling Parameter, we set it to zero.
- $\lambda = \alpha^2(n_x + \kappa) - n_x$

Prediction Step (Time Update):
Generate $2n_x + 1$ sigma points from $\hat{\mathbf{x}}_{t-1}$ and $\mathbf{P}_{t-1}$ (augmented form with process noise is common in literature, but here we show the standard non-augmented version for clarity and consistency with EKF notation):

$$\chi_{t-1}^{(0)} = \hat{\mathbf{x}}_{t-1} \tag{73}$$

$$\chi_{t-1}^{(i)} = \hat{\mathbf{x}}_{t-1} + \big(\sqrt{(n_x+\lambda)\mathbf{P}_{t-1}}\big)_i \quad i = 1, \ldots, n_x \tag{74}$$

$$\chi_{t-1}^{(i)} = \hat{\mathbf{x}}_{t-1} - \big(\sqrt{(n_x+\lambda)\mathbf{P}_{t-1}}\big)_{i-n_x} \quad i = n_x+1, \ldots, 2n_x \tag{75}$$

Propagate through the nonlinear state transition:

$$\chi_{t|i}^{-} = f\big(\chi_{t-1}^{(i)}\big), \quad i = 0, \ldots, 2n_x \tag{76}$$

Compute predicted mean and covariance:

$$\hat{\mathbf{x}}_t^{-} = \sum_{i=0}^{2n_x} W_i^{(m)}\, \chi_{t|i}^{-} \tag{77}$$

$$\mathbf{P}_t^{-} = \sum_{i=0}^{2n_x} W_i^{(c)} \left(\chi_{t|i}^{-} - \hat{\mathbf{x}}_t^{-}\right)\left(\chi_{t|i}^{-} - \hat{\mathbf{x}}_t^{-}\right)^{\top} \tag{78}$$

Update Step (Measurement Update):
Propagate sigma points through the nonlinear measurement function:

$$\zeta_{t|i}^{-} = h\big(\chi_{t|i}^{-}\big), \quad i = 0, \ldots, 2n_x \tag{79}$$

Compute predicted measurement mean, innovation covariance, and cross-covariance:

$$\hat{\mathbf{z}}_t^{-} = \sum_{i=0}^{2n_x} W_i^{(m)}\, \zeta_{t|i}^{-} \tag{80}$$

$$\mathbf{P}_{zz,t}^{-} = \sum_{i=0}^{2n_x} W_i^{(c)} \left(\zeta_{t|i}^{-} - \hat{\mathbf{z}}_t^{-}\right)\left(\zeta_{t|i}^{-} - \hat{\mathbf{z}}_t^{-}\right)^{\top} \tag{81}$$

$$\mathbf{P}_{xz,t}^{-} = \sum_{i=0}^{2n_x} W_i^{(c)} \left(\chi_{t|i}^{-} - \hat{\mathbf{x}}_t^{-}\right)\left(\zeta_{t|i}^{-} - \hat{\mathbf{z}}_t^{-}\right)^{\top} \tag{82}$$

Define the **innovation**:

$$\mathbf{v}_t = \mathbf{z}_t - \hat{\mathbf{z}}_t^{-} \tag{83}$$

Compute the **Kalman gain**:

$$\mathbf{K}_t = \mathbf{P}_{xz,t}^{-}\big(\mathbf{P}_{zz,t}^{-}\big)^{-1} \tag{84}$$

Update state mean and covariance:

$$\hat{\mathbf{x}}_t = \hat{\mathbf{x}}_t^{-} + \mathbf{K}_t\, \mathbf{v}_t \tag{85}$$

$$\mathbf{P}_t = \mathbf{P}_t^{-} - \mathbf{K}_t\, \mathbf{P}_{zz,t}^{-}\, \mathbf{K}_t^{\top} \tag{86}$$

B Particle flow and particle filter

B.1 Particle filter

Importance sampling, more general than Bootstrap particle filter

B.2 Particle flow

Previously, we have discussed how after each resampling, the information of the distribution is concentrated on the multiplicity distribution of the particles, and the diversity of the particles have been lost. If we look at the particle methods, it's clearly that we either use weights or multiplicity distribution to mimic distribution. The culprit of the particle degeneracy is the consecutive multiplication of small numbers $p(y_t|x_t^{(i)})$ will make a large number of particles irrelevant after view steps. And the observation was done independently and there is no way to control the value of $p(y_t|x_t^{(i)})$. In [11, 12], the author proposed to use the multiplicity to store the information of the distribution and invented an alternative evolution for the particles such that $p(y_t|x_t^{(i)})$ was never too small. The limitation of these two papers is that to solve the equation, which we will talk about later, they have to assume linear or nearly linearly observation model and dynamics with additive Gaussian noise. The evolution for the particles is complete different equation than underlying dynamics for the ground truth. Later, we will see the methods that generalize the equation for more general noise distribution. Let us first derive the evolution equations. First, we have the prediction step:

$$p(x_t|z_{1:t-1}) = g(x_t) \tag{87}$$

and the "observation" step,

$$p(z_t|x_t) = h(x_t) \tag{88}$$

combine both information, the updated probability for x_t should be,

$$p(x_t|z_{1:t}) \propto g(x_t)h(x_t) \tag{89}$$

The normalization will be handled implicitly by the particles. The difference between the particle flow and the particle filter is how you do the update. Since we are moving the particles directly, we demand,

$$\log p(x, \lambda) = \log g(x) + \lambda \log h(x) \tag{90}$$

we always start with $\lambda = 0$, and end with $\lambda = 1$, therefore we can assure the particle flows to the desired position. By definition, the evolution is driven entirely by the observation model. This is where the limitation of the flow model comes in.

Assuming particles evolve in pseudo-time λ according to the ordinary differential equation:

$$\frac{d\mathbf{x}}{d\lambda} = f(\mathbf{x}, \lambda) \tag{91}$$

where $f(\mathbf{x}, \lambda)$ is the flow field that transports particles from the prior distribution at $\lambda = 0$ to the posterior distribution at $\lambda = 1$. This deterministic evolution of particles leads to a corresponding evolution of the probability density $p(\mathbf{x}, \lambda)$.

The evolution of the probability density is governed by the Fokker-Planck equation with zero diffusion. Starting from the continuity equation for probability conservation, and noting that the flow is deterministic (no diffusion term), we obtain:

$$\frac{\partial p}{\partial \lambda} = -\nabla \cdot (pf) \tag{92}$$

This is the zero-diffusion Fokker-Planck equation, which expresses that the rate of change of probability density equals the negative divergence of the probability flux pf. To work with log-probabilities, we divide both sides by p and apply the product rule for divergence, $\nabla \cdot (pf) = p\nabla \cdot f + (\nabla p) \cdot f$, yielding: Recognizing that $\frac{1}{p}\frac{\partial p}{\partial \lambda} = \frac{\partial \log p}{\partial \lambda}$ and $\frac{\nabla p}{p} = \nabla \log p$, we obtain:

$$\frac{\partial \log p}{\partial \lambda} = -\nabla \cdot f - (\nabla \log p) \cdot f \tag{93}$$

From the log-homotopy definition, we have $\frac{\partial \log p}{\partial \lambda} = \log h(x)$, which leads to the fundamental PDE:

$$\boxed{\log h(x) = -\nabla \cdot f - (\nabla \log p) \cdot f} \tag{94}$$

To solve this equation, we assume an affine flow ansatz:

$$f(x, \lambda) = A(\lambda)x + b(\lambda) \tag{95}$$

where $A(\lambda)$ and $b(\lambda)$ are to be determined. The goal is to find expressions for these functions such that particles following this flow evolve from the prior distribution $g(x)$ at $\lambda = 0$ to the posterior distribution proportional to $g(x)h(x)$ at $\lambda = 1$.

We begin by computing the gradient of the log-homotopy. Using the Gaussian form for both $g(x)$ and $h(x)$, and substituting the explicit forms of the prior and likelihood into the log-homotopy:

$$\log p(x, \lambda) = -\frac{1}{2}(x - \bar{\eta}_0)^T P^{-1}(x - \bar{\eta}_0) - \frac{\lambda}{2}(z - Hx)^T R^{-1}(z - Hx) + \text{const} \tag{96}$$

Taking the gradient with respect to x:

$$\nabla_x \log p(x, \lambda) = -P^{-1}(x - \bar{\eta}_0) + \lambda H^T R^{-1}(z - Hx) \tag{97}$$

$$= -P^{-1}x + P^{-1}\bar{\eta}_0 + \lambda H^T R^{-1} z - \lambda H^T R^{-1} Hx \tag{98}$$

For the affine flow, the divergence is simply the trace of the matrix A:

$$\nabla \cdot f = \nabla \cdot (Ax + b) = \text{Tr}(A) \tag{99}$$

The dot product $(\nabla \log p) \cdot f$ expands as:

$$(\nabla \log p) \cdot f = [-P^{-1}(x - \bar{\eta}_0) + \lambda H^T R^{-1}(z - Hx)]^T [Ax + b] \tag{100}$$

$$= -(x - \bar{\eta}_0)^T P^{-1} Ax - (x - \bar{\eta}_0)^T P^{-1} b \tag{101}$$

$$+ \lambda (z - Hx)^T R^{-1} HAx + \lambda (z - Hx)^T R^{-1} Hb \tag{102}$$

The log-likelihood for the linear-Gaussian observation model can be expanded as:

$$\log h(x) = -\frac{1}{2}(z - Hx)^T R^{-1}(z - Hx) + \text{const} \tag{103}$$

$$= -\frac{1}{2} z^T R^{-1} z + z^T R^{-1} Hx - \frac{1}{2} x^T H^T R^{-1} Hx + \text{const} \tag{104}$$

A key insight for the EDH flow is that, under Gaussian assumptions, the distribution at any pseudo-time λ remains Gaussian. Specifically, the posterior at λ is:

$$p(x, \lambda) \sim \mathcal{N}(\bar{\eta}_\lambda, P_\lambda) \tag{105}$$

where the precision matrix and mean evolve according to:

$$P_\lambda^{-1} = P^{-1} + \lambda H^T R^{-1} H \tag{106}$$

$$\bar{\eta}_\lambda = P_\lambda [P^{-1}\bar{\eta}_0 + \lambda H^T R^{-1} z] \tag{107}$$

This allows us to express the gradient of the log-probability in a compact form:

$$\nabla \log p(x, \lambda) = -P_\lambda^{-1}(x - \bar{\eta}_\lambda) \tag{108}$$

Substituting this into the fundamental PDE, we obtain an alternative formulation:

$$\log h(x) = -\text{Tr}(A) + (x - \bar{\eta}_\lambda)^T P_\lambda^{-1}(Ax + b) \tag{109}$$

A more direct approach to determining $A(\lambda)$ and $b(\lambda)$ comes from requiring that particles starting from a Gaussian distribution remain Gaussian under the flow. For an affine flow $f = Ax + b$, the evolution of the mean and covariance must satisfy:

$$\frac{d\bar{\eta}_\lambda}{d\lambda} = A\bar{\eta}_\lambda + b, \quad \frac{dP_\lambda}{d\lambda} = AP_\lambda + P_\lambda A^T \tag{110}$$

From Kalman filter theory, we know that the precision matrix evolves as:

$$\frac{dP_\lambda^{-1}}{d\lambda} = H^T R^{-1} H \tag{111}$$

Using the identity $\frac{dP_\lambda}{d\lambda} = -P_\lambda \frac{dP_\lambda^{-1}}{d\lambda} P_\lambda$, we find:

$$\frac{dP_\lambda}{d\lambda} = -P_\lambda H^T R^{-1} H P_\lambda \tag{112}$$

Matching this with the covariance evolution equation $AP_\lambda + P_\lambda A^T = -P_\lambda H^T R^{-1} H P_\lambda$, and solving the resulting Riccati-type equation with boundary condition $P_0 = P$, we obtain:

$$\boxed{A(\lambda) = -\frac{1}{2} P_\lambda H^T R^{-1} H = -\frac{1}{2} P H^T (\lambda H P H^T + R)^{-1} H} \tag{113}$$

For the mean evolution, we require that $\frac{d\bar{\eta}_\lambda}{d\lambda} = H^T R^{-1}(z - H\bar{\eta}_\lambda)$ equals $A\bar{\eta}_\lambda + b$. Solving for b:

$$b(\lambda) = P_\lambda H^T R^{-1} z - A\bar{\eta}_\lambda \tag{114}$$

After algebraic manipulation, this simplifies to:

$$\boxed{b(\lambda) = (I + 2\lambda A)[(I + \lambda A) P H^T R^{-1} z + A\bar{\eta}_0]} \tag{115}$$

The EDH flow equations are derived by requiring that Gaussian distributions remain Gaussian under the flow, that the flow satisfies the Fokker-Planck equation, and that the boundary conditions $p(x, 0) = g(x)$ and $p(x, 1) \propto g(x)h(x)$ are satisfied. The resulting solution provides an **exact** method for transporting particles from the prior to the posterior in the linear-Gaussian case, avoiding the weight degeneracy issues that plague standard particle filters. As for the LEDH, the difference is that each particle has their own equation. That is because the expansion of the model are taken at their individual location instead of the average position.

B.3 SDE

B.4 Particle flow filter (kernel method)

Our problem is the same as particle filter particle flow, move the particles to the desired location, without touching the weights, so that we get the target probability distribution $p(x_t|y_{1:t}) \propto p(y_t|x_t)p(x_t|y_{1:t-1})$.

B.4.1 Optimal Transport via Gradient Flow

Define target posterior $\pi(x) := p(x_t|y_{1:t})$ and intermediate density $q_\lambda(x)$ at pseudo-time $\lambda \in [0, 1]$. With $q_0 = p(x_t|y_{1:t-1})$ (prior), $q_1 = \pi$ (posterior).

$$\frac{dx_\lambda}{d\lambda} = v_\lambda(x_\lambda) \tag{116}$$

Choose velocity v_λ to minimize KL divergence as fast as possible:

$$D_{KL}(q_\lambda \| \pi) = \int q_\lambda(x) \log \frac{q_\lambda(x)}{\pi(x)} dx \tag{117}$$

we need to formulate the above equation into an optimization problem. But the intuition of this method is clear, you want to evolve the point along a "straight" direction to the desire point.

B.4.2 Deriving Optimal Velocity via Kernel Restriction

Under velocity field v, density evolves via Liouville equation:

$$\frac{\partial q_\lambda}{\partial \lambda} = -\nabla_x \cdot (q_\lambda v) \tag{118}$$

Time derivative of D_{KL}:

$$\frac{d}{d\lambda}D_{KL} = \int \frac{\partial q_\lambda}{\partial \lambda}\left[\log q_\lambda(x) - \log \pi(x) + 1\right] dx \tag{119}$$

$$= -\int \nabla_x \cdot (q_\lambda v)\left[\log q_\lambda(x) - \log \pi(x) + 1\right] dx \tag{120}$$

Using $\nabla_x q_\lambda = q_\lambda \nabla_x \log q_\lambda$ and integrating by parts:

$$\frac{d}{d\lambda}D_{KL} = -\int q_\lambda \left[\nabla_x \log \pi(x) \cdot v + \nabla_x \cdot v\right] dx \tag{121}$$

The derivative with respect to the evolution parameter gives the vector field along the trajectory. To make things computatble, we restrict v to functions generated by kernel $K(x, x')$. Any such function has form:

$$v(x) = \int \alpha(x')K(x', x)dx' \tag{122}$$

for some coefficient function $\alpha(x')$. Substitute this into directional derivative.

$$\frac{d}{d\lambda}D_{KL} = -\int q_\lambda(x)\left[\nabla_x \log \pi(x) \cdot \int \alpha(x')K(x', x)dx' + \nabla_x \cdot \int \alpha(x')K(x', x)dx'\right] dx \tag{123}$$

Interchange integrals and use integration by parts:

$$\frac{d}{d\lambda}D_{KL} = \int \alpha(x) \cdot \phi(x)dx \tag{124}$$

where

$$\phi(x) = -\int q_\lambda(x')\left[K(x, x')\nabla_{x'} \log \pi(x') + \nabla_{x'}K(x, x')\right] dx' \tag{125}$$

We have $\frac{d}{d\lambda}D_{KL} = \langle \alpha, \phi \rangle$ (inner product in function space).

To minimize this quantity (make KL decrease fastest), choose α antiparallel to ϕ. By Cauchy-Schwarz, the minimum is achieved when:

$$\alpha(x) = -\phi(x) \tag{126}$$

Therefore, optimal velocity field:

$$v_\lambda(x) = \int \alpha(x')K(x', x)dx' \tag{127}$$

$$= -\int \phi(x')K(x', x)dx' \tag{128}$$

By the reproducing property of the kernel, this simplifies to:

$$\boxed{v_\lambda(x) = \mathbb{E}_{x' \sim q_\lambda}\left[K(x, x')\nabla_{x'} \log \pi(x') + \nabla_{x'}K(x, x')\right]} \tag{129}$$

Monte Carlo approximation with particles $\{x_\lambda^{(i)}\}_{i=1}^{N_p} \sim q_\lambda$:

$$v_\lambda(x) \approx \frac{1}{N_p}\sum_{i=1}^{N_p}\left[K(x_\lambda^{(i)}, x)\nabla \log \pi(x_\lambda^{(i)}) + \nabla K(x_\lambda^{(i)}, x)\right] \tag{130}$$

Posterior gradient:

$$\nabla \log \pi(x) = \nabla \log p(y_t|x) + \nabla \log p(x|y_{1:t-1}) \tag{131}$$

First term = likelihood gradient (known). Second term = prior gradient (approximated from ensemble).

Update rule:

$$x_{\lambda+\epsilon}^{(j)} = x_\lambda^{(j)} + \frac{\epsilon}{N_p}\sum_{i=1}^{N_p}\left[K(x_\lambda^{(i)}, x_\lambda^{(j)})\nabla \log \pi(x_\lambda^{(i)}) + \nabla K(x_\lambda^{(i)}, x_\lambda^{(j)})\right] \tag{132}$$

All particles maintain equal weight $w^{(i)} = 1/N_p$ throughout. No resampling needed.

Generality: Works for any differentiable likelihood $p(y_t|x_t)$ - not limited to Gaussian noise.

B.4.3 Scalar Kernel

Standard choice: isotropic Gaussian

$$K(x, x') = \exp\left(-\frac{1}{2\alpha\sigma^2}\|x - x'\|^2\right), \quad \alpha \sim n_x \tag{133}$$

Gradient: $\nabla_x K(x, x') = -\frac{1}{\alpha\sigma^2}(x - x')K(x, x')$
Single scalar $K(x, x')$ weights all components identically.

B.4.4 Matrix-Valued Kernel

Measure distance independently per component:

$$\mathbf{K}(x, x') = \text{diag}\left[K^{(1)}(x, x'), \ldots, K^{(n_x)}(x, x')\right] \tag{134}$$

where

$$K^{(d)}(x, x') = \exp\left(-\frac{(x^{(d)} - x'^{(d)})^2}{2\alpha(\sigma^{(d)})^2}\right), \quad \alpha = \frac{1}{N_p} \tag{135}$$

Component-wise repulsion:

$$[\nabla_x \cdot \mathbf{K}(x, x')]^{(d)} = -\frac{x^{(d)} - x'^{(d)}}{\alpha(\sigma^{(d)})^2} K^{(d)}(x, x') \tag{136}$$

Observed dimensions maintain diversity regardless of unobserved dimension spread.

References

[1] Yudong Feng and Ashis Gangopadhyay. On a semiparametric stochastic volatility model, 2025.

[2] Luigi Catello, Ludovica Ruggiero, Lucia Schiavone, and Mario Valentino. Hidden markov models for stock market prediction, 2023.

[3] Olivier Guéant and Jiang Pu. Mid-price estimation for european corporate bonds: a particle filtering approach. *Market Microstructure and Liquidity*, 4(01n02):1950005, 2018.

[4] Simo Särkkä. *Bayesian Filtering and Smoothing*. Cambridge University Press, 2013.

[5] Shida Jiang, Junzhe Shi, and Scott Moura. A new framework for nonlinear kalman filters, 2025.

[6] Ienkaran Arasaratnam and Simon Haykin. Cubature kalman filters. *IEEE Transactions on automatic control*, 54(6):1254–1269, 2009.

[7] Geir Evensen. The ensemble kalman filter: Theoretical formulation and practical implementation. 2003.

[8] Omar Al-Ghattas, Jiajun Bao, and Daniel Sanz-Alonso. Ensemble kalman filters with resampling, 2024.

[9] Guy Revach, Nir Shlezinger, Xiaoyong Ni, Adrià López Escoriza, Ruud J. G. van Sloun, and Yonina C. Eldar. Kalmannet: Neural network aided kalman filtering for partially known dynamics. *CoRR*, abs/2107.10043, 2021.

[10] Hassan Mortada, Cyril Falcon, Yanis Kahil, Mathéo Clavaud, and Jean-Philippe Michel. Recursive kalmannet: Deep learning-augmented kalman filtering for state estimation with consistent uncertainty quantification. *arXiv preprint arXiv:2506.11639*, 2025. eess.SP.

[11] Fred Daum, Jim Huang, and Arjang Noushin. Exact particle flow for nonlinear filters. In *Signal processing, sensor fusion, and target recognition XIX*, volume 7697, pages 92–110. SPIE, 2010.

[12] Fred Daum and Jim Huang. Particle degeneracy: root cause and solution. In *Signal Processing, Sensor Fusion, and Target Recognition XX*, volume 8050, pages 367–377. SPIE, 2011.

[13] Tao Ding and Mark J. Coates. Implementation of the daum-huang exact-flow particle filter. In *2012 IEEE Statistical Signal Processing Workshop (SSP)*, pages 257–260. IEEE, 2012.

[14] Liyi Dai and Fred Daum. A new parameterized family of stochastic particle flow filters, 2021.

[15] Liyi Dai and Frederick E. Daum. Stiffness mitigation in stochastic particle flow filters, 2021.

[16] Yunpeng Li and Mark Coates. Particle filtering with invertible particle flow, 2017.

[17] Manuel Pulido and Peter Jan van Leeuwen. Sequential monte carlo with kernel embedded mappings: The mapping particle filter. 2019.

[18] Chih-Chi Hu and Peter Jan Van Leeuwen. A particle flow filter for high-dimensional system applications. 2021.

[19] Sebastian Reich. A nonparametric ensemble transform method for bayesian inference. *SIAM Journal on Scientific Computing*, 35(4):A2013–A2024, 2013.

[20] Adrien Corenflos, James Thornton, George Deligiannidis, and Arnaud Doucet. Differentiable particle filtering via entropy-regularized optimal transport, 2021.

[21] Shreyas Chaudhari, Srinivasa Pranav, and José M. F. Moura. Gradnetot: Learning optimal transport maps with gradnets. *arXiv preprint arXiv:2507.13191*, 2025. v2.

[22] Michael Zhu, Kevin P. Murphy, and Rico Jonschkowski. Towards differentiable resampling. *arXiv preprint arXiv:2004.11938*, 2020.

[23] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam R. Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 3744–3753. PMLR, 2019.

[24] Xiongjie Chen and Yunpeng Li. An overview of differentiable particle filters for data-adaptive sequential bayesian inference. *Foundations of Data Science*, 2023. Also available as arXiv:2302.09639.

[25] Brandon Amos. Tutorial on amortized optimization: Learning to optimize over continuous spaces. *arXiv preprint arXiv:2202.00665*, 2022. Version 5, October 2025.

[26] Brandon Amos, Giulia Luise, Samuel Cohen, and Ievgen Redko. Meta optimal transport. In *Proceedings of the 40th International Conference on Machine Learning*, PMLR 202, Honolulu, Hawaii, USA, 2023. arXiv:2206.05262v2.

[27] Christophe Andrieu, Arnaud Doucet, and Roman Holenstein. Particle markov chain monte carlo methods. 2010.

[28] Nicholas Metropolis, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953.

[29] Radford M. Neal. Mcmc using hamiltonian dynamics, 2012.

[30] Simon Duane, A.D. Kennedy, Brian J. Pendleton, and Duncan Roweth. Hybrid monte carlo. *Physics Letters B*, 195(2):216–222, 1987.

[31] Rico Jonschkowski, Divyam Rastogi, and Oliver Brock. Differentiable particle filters: End-to-end learning with algorithmic priors. In *Robotics: Science and Systems XIV*, 2018.

[32] Fredrik Lindsten, Michael I. Jordan, and Thomas B. Schön. Particle gibbs with ancestor sampling. *Journal of Machine Learning Research*, 15:2145–2184, 2014.